



Language Tool

Documentation

Creator: Lucas Gomes Cecchini

Online Name: AGAMENOM

Overview

This document will help you to use **Assets Language Tool** for **Unity**.

With it you can translate your game into several languages, automatic selection of languages (if you have a file available) and a simple text file to interpret, making it possible for anyone to translate the game into a new language.

It also has simple features to translate images and sound.

Instructions

You can get more information from the Playlist on YouTube:

https://youtube.com/playlist?list=PL5hnfx09yM4JkAyxrZWaFjhO3NMWxP_1F

Script Explanations

Canvas Manager

This script is a **utility system for saving, rebuilding, and validating UI layouts** made with Unity's Canvas system. Think of it as a way to **turn a visual UI hierarchy into data... and later rebuild that UI from that data**.

I'll break it down clearly: variables, methods, and how you would actually use it.

Core Idea

The script revolves around a concept called a **"CanvasStructure"** (a data container, defined elsewhere).

It supports three main workflows:

1. **Extract** → Read an existing UI and convert it into data
2. **Apply** → Take data and apply it to an existing UI
3. **Create** → Build a brand-new UI from data

Variables

Ignored Prefixes

ignoredPrefixes

A list of name patterns like:

- "TMP SubMeshUI"
- "Dropdown List"
- "Blocker"

Purpose:

Some UI systems (like TextMeshPro) create **internal helper objects**. These are **ignored** so they don't pollute your saved structure.

Public Methods (Main Features)

1. ExtractCanvasData

What it does:

Reads a Canvas object and converts it into structured data.

Step-by-step:

1. Creates a new empty structure
2. Checks for **duplicate object names**
3. Stores the Canvas name
4. Temporarily **activates all children** (even hidden ones)
5. Extracts:
 - Transform data (position, size, anchors)
 - Canvas settings
 - Scaling settings
 - Raycasting settings
6. Walks through the hierarchy and records all paths
7. Restores original active states

Why it matters:

You get a **complete snapshot of the UI layout**, even inactive elements.

2. ApplyCanvasData

What it does:

Takes saved data and applies it to an **existing Canvas**.

Behavior:

- Updates:
 - Layout (position, anchors, size)
 - Canvas settings
- Only modifies objects that already exist
- Does **NOT create missing objects**

Important detail:

If something is missing in the hierarchy, it logs a warning instead of creating it.

👉 This is for **updating**, not rebuilding.

3. CreateCanvasFromStructure

What it does:

Builds a **new Canvas from scratch** using stored data.

Step-by-step:

1. Validates the structure
2. Creates a root object (Canvas + required components)
3. Recreates hierarchy layer by layer
4. For each UI element:
 - Creates object if it doesn't exist
 - Applies transform data
 - Adds visual helpers:
 - Random color (for debugging)
 - Outline effect

Result:

You get a **fully reconstructed UI layout**.

Hierarchy Processing

GenerateCanvasLayers

What it does:

Breaks the UI into **paths from root → leaf**

Example:

Canvas

|— Panel

| |— Button

Becomes:

["Panel", "Button"]

Each path also stores transform data.

Why:

This allows **precise reconstruction of hierarchy structure**.

ContainsDuplicateSiblings

What it does:

Checks if two objects at the same level have the same name.

Why:

The system relies on names to rebuild hierarchy.

If duplicates exist:

👉 Reconstruction becomes ambiguous → so it blocks extraction.

🔧 Utility Methods

ConvertToRectTransformData

What it stores:

- Position
- Rotation
- Scale
- Anchors
- Size
- Pivot

👉 Basically everything needed to rebuild layout.

ActivateAllChildren

What it does:

Temporarily enables all objects in the hierarchy.

Why:

Inactive objects normally wouldn't be detected → this ensures **nothing is missed**.

FindChildByName

What it does:

Searches for a direct child with a specific name.

Important:

- Only checks immediate children
- Not recursive

IsIgnoredName

What it does:

Checks if a name starts with any ignored prefix.

Used in:

- Extraction
- Validation
- Traversal

Metadata Handling

PopulateCanvasMetadata

Extracts:

- Transform data
- Canvas settings (render mode, sorting, etc.)
- Scaling behavior
- Raycasting rules

 Converts components → data

UpdateCanvasMetadata

Applies:

- Transform settings
- Canvas configuration
- Scaling and interaction settings

 Converts data → components

How You Would Use This

Example 1 — Save UI Layout

1. Take an existing Canvas

2. Call extraction
3. Store the result (file, memory, etc.)

👉 Now you have a reusable UI blueprint

✅ Example 2 — Rebuild UI

1. Load stored structure
2. Call creation
3. A new Canvas is generated

👉 Useful for:

- Dynamic UI generation
- Procedural interfaces
- UI templates

✅ Example 3 — Update Existing UI

1. Have a Canvas already in scene
2. Apply stored structure

👉 Useful for:

- Fixing layouts
- Syncing UI versions
- Applying presets

⚠️ Important Constraints

1. Unique Names Required

Every sibling must have a unique name.

Bad:

Button

Button

Good:

Button_Play

2. Structure Consistency

Each hierarchy path must match:

- Same number of names
- Same number of transform entries

3. Ignored Objects

Internal UI objects are skipped automatically.

Big Picture

This script is basically:

 A **serialization system for UI hierarchies**

It lets you:

- Save UI layouts as data
- Rebuild them exactly
- Apply them selectively

Why this is powerful

- You can version UI layouts like assets
- You can generate UI dynamically at runtime
- You can create tools like visual UI editors
- You decouple UI design from scenes

Font And Alignment Utility / Font And Alignment Utility TMP

This script is a **small but important bridge between your localization system and TextMeshPro.**

It solves two common problems in multilingual UI:

1. **How to select fonts dynamically** (depending on language)
2. **How to store alignment in a simple format** (like numbers in files such as TSV/JSON)

Core Idea

Instead of directly using font assets and alignment enums everywhere, this utility:

- Uses **indexes** to identify fonts
- Uses **integers** to represent alignment

Why?

Because numbers are easier to:

- Save in files
- Serialize
- Share between systems

Variables

This script is mostly stateless, but it depends on one external system:

Language Settings (external dependency)

LoadLanguageSettings()

This comes from your LanguageFileManager.

It provides:

- A list of fonts (TMPFontList)

Think of it as:

A centralized database of fonts for all languages.

Font Methods

1. GetFontByIndex

What it does:

Returns a font based on a **number**.

How it works:

1. Loads the font list from language settings
2. Uses the given number to pick a font
3. Returns the corresponding font

Important detail:

- The index starts at **1 (not 0)**

Example idea:

If your font list is:

1 → EnglishFont

2 → JapaneseFont

3 → ArabicFont

Calling:

GetFontByIndex(2)

👉 Returns: JapaneseFont

If invalid:

- Returns nothing (null)

2. GetFontIndex

What it does:

Finds the **number (index)** of a given font.

How it works:

1. Loads the font list
2. Loops through it
3. Compares each font
4. Returns its position

Example:

If you pass:

ArabicFont

👉 It returns:

3

If not found:

- Returns **0**

📐 Alignment Conversion

This is the most important part for data-driven UI.

🔗 Why this exists

Text alignment in TextMeshPro is complex (many options).

But your system likely stores data like:

"alignment": 8

👉 This script converts between:

- **Integer (8)**
- **Actual alignment setting (Center)**

3. ConvertToTextAnchor

What it does:

Converts a number → alignment setting

Example:

8 → Center

1 → TopLeft

15 → BottomRight

Behavior:

- Uses a mapping table (switch logic)
- Each number corresponds to a specific alignment

If invalid:

- Logs a warning
- Defaults to **TopLeft**

4. ConvertToAlignmentCode

What it does:

Converts alignment setting → number

Example:

Center → 8

TopLeft → 1

Bottom → 14

If unknown:

- Logs a warning
- Returns **0 (or default)**

How You Would Use This

Example 1 — Loading UI from Data

Imagine your TSV file contains:

Text: "Hello"

FontIndex: 1

Alignment: 8

Process:

1. Get font using index
2. Convert alignment number
3. Apply both to the text component

👉 Result:

- Correct font
- Correct alignment
- Language-specific UI

✅ Example 2 — Saving UI to File

When exporting UI:

1. Take the current font
2. Convert it to index
3. Convert alignment to number
4. Save values

👉 Now your UI becomes **data-driven**

✅ Example 3 — Language Switching

When changing language:

1. Replace font using index mapping
2. Keep alignment consistent via numbers

👉 This is especially useful for:

- Arabic (RTL)
- Japanese (different spacing)
- Dynamic UI adjustments

⚠ Important Details

1. 1-Based Indexing

Fonts start at:

1 instead of 0

👉 This is intentional for:

- Cleaner data in files
- Avoiding “0 = valid value” confusion

2. Alignment Range

Supported values:

1 → 37

These cover:

- Top / Middle / Bottom
- Left / Center / Right
- Special alignments (Baseline, Midline, Capline, etc.)

3. Safe Fallbacks

If something goes wrong:

- Font → returns nothing
- Alignment → defaults safely

👉 Prevents crashes

🧠 Big Picture

This script is a **conversion layer** between:

📄 **Data (TSV / JSON / etc.)**

- Numbers (font index, alignment code)

🎨 **UI (TextMeshPro)**

- Font assets
- Alignment settings

💡 Why this is useful

- Makes your UI fully **data-driven**

- Works perfectly with your TSV system
- Supports **multilingual projects**
- Keeps files simple and readable
- Avoids hard references to assets

ID Exists Attribute

This script is an **editor validation tool**.

Its purpose is simple but very useful:

👉 **Prevent duplicate IDs in your project by warning you directly in the Inspector.**

It does this by reading a saved JSON file and checking if the number you typed is already being used somewhere else.

🧠 Core Idea

Normally, Unity lets you type any number into a field.

This script adds intelligence:

- You mark a number field with a special attribute
- The Inspector automatically checks:
 - "Is this ID already used?"
- If yes:
 - The field turns **yellow**
 - A **warning message appears**

🏷️ Attribute (IDExistsAttribute)

What it is

A **marker** you place on integer fields.

Example usage (conceptually):

IDExists → checks component IDs

IDExists(true) → checks canvas IDs

Variable

searchCanvas

A simple boolean:

- false → check **component IDs**
- true → check **canvas IDs**

👉 This lets the same system validate different types of IDs.

Constructor

What it does

When you use the attribute:

- If you don't pass anything → it checks components
- If you pass true → it checks canvases

🎨 Custom Drawer (IDExistsDrawer)

This is where the real behavior happens.

It controls how the field is drawn in the Inspector.

📁 Constants & Cached Data

FilePath

ProjectSettings/LanguageFileData.json

👉 This is the file where all IDs are stored.

cachedData

Stores the parsed content of the JSON file.

👉 Prevents reading the file every frame.

lastFileWriteTime

Stores the last time the file was modified.

👉 Used to detect when the file changes.

🖥 Inspector Rendering

OnGUI

What it does

This runs every time the Inspector draws the field.

Step-by-step:

1. Get the attribute (to know if it's canvas or component)
2. Save the current UI color
3. Check if the ID already exists
4. If it exists:
 - Change field color to **yellow**
5. Draw the number field
6. If duplicated:
 - Show a warning box below
7. Restore original color

Result in the Inspector

If ID is unique:

[12]

If ID is duplicated:

[12] (yellow)

⚠ The component ID '12' is already in use!

GetPropertyHeight

What it does

Controls how much vertical space the field uses.

Behavior:

- Normal → single line
- If duplicated → adds space for warning box

👉 Prevents UI overlap.

Validation Logic

CheckIDExists

This is the **core validation method**.

Step-by-step:

1. Check if the JSON file exists
→ If not, nothing to validate
2. Get last modified time of the file
3. If file changed:
 - Reload it
 - Parse JSON into memory
4. Depending on mode:
 - Check canvas list OR component list
5. Return:
 - true → ID already exists
 - false → safe to use

Important detail

It uses **caching + timestamp comparison**

👉 This avoids:

- Constant file reading
- Performance issues in the editor

Data Source (JSON)

The script expects something like:

componentSave → list of component IDs

canvasSave → list of canvas IDs

Each entry contains:

- an ID value

How You Use This

✅ Step 1 — Add Attribute

You attach it to an integer field:

component ID → normal check

canvas ID → special check

✅ Step 2 — Work in Inspector

When editing:

- Type a number
- Script checks automatically

✅ Step 3 — Get Feedback

If ID is already used:

- Field turns yellow
- Warning appears

👉 You immediately know there's a conflict

💡 Real Use Case

This is extremely useful for systems like:

- Localization (your case)
- UI referencing

- Data-driven systems
- Save/load systems

Example Problem Without This

You accidentally reuse ID 25:

- Two UI elements share same ID
- System loads wrong text
- Bugs appear randomly

With This Script

- You type 25
- Inspector instantly warns you
- You fix it before it becomes a bug

Important Behaviors

1. Works Only in Editor

This system:

- Does NOT run in builds
- Only helps during development

2. Only Checks Integers

If the field is not a number:

- It ignores validation

3. Safe Fallback

If JSON file is missing:

- No validation happens
- No errors thrown

4. Smart Performance

- File is only reloaded if modified
- Otherwise uses cached data

Big Picture

This script is:

 A **real-time validation system for IDs inside the Unity Inspector**

In one sentence

 It prevents ID conflicts by checking your input against a saved database and warning you instantly inside the editor.

Language Class Group

This script is not a “behavior” by itself — it’s a **data model layer**.
Think of it as the **foundation that every other system (like your CanvasManager, language tools, TMP utilities, etc.) depends on**.

It defines **how information is stored, organized, and transferred** for:

- Localization (languages, texts, IDs)
- UI layout (Canvas, RectTransform, scaling, raycasting)
- Editor tools (saving/loading JSON, validation, etc.)

Big Picture

The script is divided into logical groups:

1. **Language data (texts, IDs, metadata)**
2. **Canvas/UI structure (layout + hierarchy)**
3. **Component settings (Canvas, Scaler, Raycaster)**
4. **Runtime/editor interaction (events, options)**
5. **File manager storage (what gets saved to JSON)**

Everything is marked as *serializable*, meaning:

 It can be saved to files (like JSON) and reconstructed later.

1. Language Structures

These define how **languages and texts** are represented.

◆ LanguageAvailable

Represents a **language option in your system**.

Variables:

- **culture** → Standard language code (like "en-US", "pt-BR").
- **nativeName** → Name in its own language (e.g., "Português").
- **name** → Display name (e.g., "Portuguese").
- **isAvailable** → Whether this language can be selected.
- **columnIndex** → Links this language to a column in a TSV table.

Usage:

Used to:

- Populate dropdowns
- Decide which column of text to read from

◆ IdData

Simple pair:

- **iD** → unique identifier
- **text** → the localized text

Usage:

Basic "ID → text" mapping.

◆ IdMetaData

Stores **extra configuration for a text ID**.

Variables:

- **iD** → which text this belongs to
- **alignment** → how text is aligned
- **fontSize** → size
- **fontListIndex** → which font to use
- **componentType** → what UI system uses it (Text, TMP, etc.)

Usage:

Controls **how text looks**, not just what it says.

2. Canvas & UI Structures

These define how UI layouts are saved and rebuilt.

◆ CanvasDataList

Represents a **single UI object inside a canvas**.

Variables:

- Identity:
 - **gameObjectName**
 - **instanceID**
 - **gameObjectEnable**
- Transform:
 - **localPosition**
 - **localRotation**
 - **localScale**
- Layout:
 - **anchorMin / anchorMax**
 - **anchoredPosition**
 - **sizeDelta**
 - **pivot**

Usage:

Stores the full state of a UI element.

◆ CanvasStructure

This is the **main container for an entire UI canvas**.

Variables:

- **canvasName** → root object name
- **canvasLayers** → hierarchy paths (very important)
- **rectTransform** → root layout
- **canvas** → rendering settings
- **canvasScaler** → scaling behavior
- **graphicRaycaster** → interaction settings

Usage:

This is what gets:

- Extracted from a scene
- Saved to JSON
- Rebuilt later

◆ CanvasLayers

Represents a **single hierarchy path**.

Variables:

- **CanvasObjectsLayers** → array of names (path like: Panel → Button → Text)
- **rectTransforms** → layout data for each step

Usage:

Instead of storing a tree, the system stores **paths**.
This makes reconstruction deterministic and easier.

◆ RectTransformData

A **pure data version of UI layout**.

Variables:

- Position, rotation, scale
- Anchors
- Size
- Pivot

Usage:

Used everywhere to:

- Save layouts
- Reapply layouts

📦 3. Canvas Component Data

These store settings of Unity UI components.

◆ CanvasData

Controls how the canvas behaves visually.

Variables:

- **renderMode** → Screen space / world space
- **planeDistance** → distance from camera
- **pixelPerfect**
- **sorting options**
- **shader settings**

Usage:

Ensures the canvas looks the same after loading.

◆ CanvasScalerData

Controls how UI scales.

Variables:

- Scale mode
- Reference resolution
- DPI settings
- Match width/height

Usage:

Critical for:

- Responsive UI
- Multi-resolution support

◆ GraphicRaycasterData

Controls UI interaction.

Variables:

- **ignoreReversedGraphics**
- **blockingObjects**
- **blockingMask**

Usage:

Defines how clicks/touches interact with UI.

4. Language Components & Runtime Interaction

These are used by gameplay/UI systems.

◆ LanguageOptions

Represents a **language choice in UI**.

Variables:

- **text** → label
- **sprite** → icon (flag, etc.)
- **iD** → linked localization ID

Usage:

Dropdowns, menus, selectors.

◆ ScriptText

Connects **text to logic**.

Variables:

- **iD**
- **text**
- **targetScripts** → event triggered with the text

Usage:

Example:

- When language changes → event sends new text to systems

◆ LanguageLines

Represents a **line of dialogue or UI text**.

Variables:

- **iD**
- **text**
- **translateText** → whether it should be localized

Usage:

Useful for:

- Mixing static + dynamic text
- Skipping translation when needed



5. Language File Manager Data

This is what gets **saved to disk (JSON)**.

◆ LanguageForEditingSave

Represents editable text data.

Variables:

- **iD**
- **text**
- **alignment**
- **fontSize**
- **fontListIndex**
- **textContext**
- **componentType**

Usage:

Used in editor tools to modify localization entries.

◆ CanvasForEditingSave

Stores a canvas snapshot.

Variables:

- **canvasID**
- **textContext**
- **json** → serialized CanvasStructure

Usage:

Lets you:

- Save UI layouts
- Reload them later

◆ LanguageFileManagerWindowData

This is the **main save file structure**.

Variables:

- **languageForEditing**
- **componentSave** → all text entries
- **canvasSave** → all canvas layouts
- **availableLanguages**
- **firstTime**
- **idIndex** → next ID generator
- **showTextData** / **showCanvasData** → UI toggles
- **filesSaved**

Usage:

This is the **central database** for your localization system.

◆ ManagerLanguageIdData

Simple helper structure:

- **iD**
- **text**
- **textContext**

Usage:

Used for managing and referencing IDs in tools.

⚙ How Everything Works Together

Flow Example:

1. You create text with an **ID**
2. That ID is stored in:
 - **LanguageForEditingSave**
3. UI elements reference that ID
4. Metadata (**IdMetaData**) controls how it looks
5. Canvas layout is saved using:
 - **CanvasStructure**
6. Everything is stored in:
 - **LanguageFileManagerWindowData** (JSON)

✖ Why This Design Is Powerful

✅ Separation of concerns

- Text ≠ layout ≠ rendering settings

✅ Fully serializable

- Everything can be saved/loaded

✅ Deterministic UI reconstruction

- Uses hierarchy paths instead of fragile references

✅ Scalable localization

- IDs + metadata allow flexible control

🧠 Simple Analogy

Think of this script as:

- 📄 **Database schema** (how data is structured)
- 🏠 **Blueprint** (how UI is rebuilt)
- 🌐 **Dictionary system** (how text is localized)

🔧 Typical Usage Scenario

1. Designer builds UI in Unity
2. System extracts:
 - Layout → CanvasStructure
 - Text → IDs
3. Everything saved to JSON
4. At runtime:
 - UI is rebuilt
 - Text is applied per language
 - Fonts & alignment are applied

Language Editor Utilities

This script is a **toolbox for the Unity Editor**, specifically built to support your localization system.

It doesn't define gameplay behavior — instead, it provides **helper functions** used by editor windows, TSV tools, and your language manager.

Big Picture

This class groups together **four main responsibilities**:

1. **Finding assets (prefabs)**
2. **Creating GUI styles (editor visuals)**
3. **Working with TSV tables (read/write/sync)**
4. **Opening and feeding data into editor windows**
5. **Building and managing localization tables**

It acts like a **bridge between your data (TSV, JSON, IDs)** and the **Unity Editor UI**.

Constants

fileData

- Path: "ProjectSettings/LanguageFileData.json"

What it means:

This is where your **main localization database is stored**.

Usage:

Used when:

- Saving language data
- Loading existing data

1. Prefab Utilities

FindPrefabByName

What it does:

Searches the Unity project for a prefab by name.

How it works:

1. Searches all prefab assets
2. Filters only those inside a specific folder (Language Tool/Prefab)
3. Compares names ignoring case and file extension

4. Returns the matching object

Important behavior:

- If nothing is found → logs an error

Usage:

Used when:

- You want to dynamically load UI elements or templates by name
- Editor tools need to instantiate prefabs automatically

2. GUI Style Utilities

These functions control how things look in the editor.

◆ CreateLabelStyle

Variables:

- **fontSize** → size of the text
- **bold** → whether text is bold
- **centerAlignment** → center or left alignment

What it does:

Creates a **custom label style**.

Usage:

Used when:

- Drawing titles
- Custom inspector labels
- Editor window UI

◆ CreateCustomButtonStyle

Variables:

- **fontSize**

What it does:

Creates a **styled button**:

- Bold text
- White normal color
- Red on hover
- Centered

Usage:

Used for:

- Custom editor buttons
- Highlighted actions



3. TSV Utilities

This is one of the most important sections.

It handles **reading, writing, validating, and syncing localization tables**.



IsInvalidTSV

What it checks:

- File path is empty
- File doesn't exist
- File is not .tsv

Returns:

- true → invalid
- false → valid

Usage:

Before loading a file, to avoid crashes or invalid data.



ExtractIDsEditor

Purpose:

Reads a TSV table and extracts localization data.

Inputs:

- Table data

- Language list
- Culture code (e.g., "en-US")

What it does:

1. Validates table structure
2. Finds the column for the selected language
3. Iterates rows starting from row 4
4. Reads:
 - Context
 - ID
 - Text
5. Converts them into structured objects

Output:

List of:

- ID
- Text
- Context

Usage:

Used when:

- Loading localization data from TSV into the editor system

◆ InsertIDsEditor**Purpose:**

Writes localization data back into a TSV table.

What it does:

1. Validates inputs
2. Finds correct language column
3. Expands table if needed
4. Ensures all rows/columns exist
5. Writes:
 - Context
 - ID
 - Translated text

Important detail:

It **auto-resizes the table**.

Usage:

Used when:

- Saving edited text back into TSV

◆ SyncTableWithIds

Purpose:

Keeps the TSV table synchronized with a list of IDs.

What it does:

1. Reads all existing IDs in the table
2. Compares with new ID list
3. Removes:
 - IDs no longer used
4. Adds:
 - Missing IDs
5. Keeps everything sorted

Key idea:

👉 The table becomes a **mirror of your current ID list**

Usage:

Used when:

- IDs change dynamically
- You want to avoid orphan or missing entries

🏠 4. Table Builders

These functions **create or modify table structures**.

◆ BuildTableFromAvailableLanguages

Purpose:

Creates a **new TSV table structure from scratch**

What it builds:

- 4 header rows:

1. Culture codes
 2. Native names
 3. Display names
 4. Availability flags
- First columns:
 - Context
 - ID

Important:

- Column count is based on language list
- Automatically fills metadata

Usage:

Used when:

- Creating a new localization table
- Initializing project setup

◆ **ReplaceTopRows**

Purpose:

Replaces only the **header section** of a table.

What it does:

- Keeps existing data
- Replaces first 4 rows

Usage:

Useful when:

- Updating language configuration
- Keeping translations but changing metadata

□ **5. Editor Window Integration**

These methods connect your data to your **custom editor window**.

◆ **OpenEditorWindowWithComponent**

Purpose:

Opens the editor and injects a text component.

What it does:

1. Creates a data object (ID, text, font, etc.)
2. Opens the editor window
3. Waits one frame (important)
4. Adds the component to the window

Why delay is used:

Because the window needs time to initialize.

Usage:

Used when:

- Clicking UI elements in the scene
- Sending data to the language editor

◆ **OpenEditorWindowWithCanvas**

Purpose:

Same idea, but for **canvas data**

What it does:

1. Creates canvas data object (ID + JSON)
2. Opens editor
3. Injects canvas next frame

Usage:

Used when:

- Editing UI layouts linked to localization

 6. Default Database Creation

◆ **CreateLanguageDataBase**

Purpose:

Creates a **complete default language table**

What it contains:

- Hundreds of culture codes
- Native names
- Display names
- Default values

Structure:

- Row 0 → culture codes
- Row 1 → native names
- Row 2 → display names
- Row 3 → availability flags

Usage:

Used when:

- First-time setup
- Generating a full multilingual database instantly

How Everything Works Together

Example Flow:

1. Create table:
→ BuildTableFromAvailableLanguages
2. Load TSV:
→ ExtractIDsEditor
3. Modify text in editor
4. Save changes:
→ InsertIDsEditor
5. Sync IDs:
→ SyncTableWithIds
6. Open editor UI:
→ OpenEditorWindowWithComponent

Key Design Ideas

Editor-first design

Everything is optimized for Unity Editor workflows.

Safe data handling

- Validation everywhere
- Prevents broken tables

Dynamic resizing

Tables adapt automatically.

✅ Separation of concerns

- TSV logic
- GUI
- Prefabs
- Editor windows

All handled independently.

🧩 Simple Analogy

Think of this script as:

- 🧰 **Toolkit** → functions you call when needed
- 📄 **Translator helper** → converts TSV ↔ structured data
- 🌉 **Editor bridge** → sends data into UI windows
- 🏗️ **Builder** → creates and fixes tables

Language File Manager

This script is the **core manager of your localization system**. It acts as the bridge between:

- Files on disk (TSV tables in StreamingAssets).
- Runtime settings (selected language, available languages).
- Structured data used by your UI and systems (IDs, texts, metadata).

Instead of being tied to visuals, it's a **data pipeline + cache + parser**.

🧠 Overall Concept

The script does 5 main things:

1. **Loads configuration (LanguageSettingsData).**
2. **Remembers the selected language (culture code).**
3. **Finds and manages file paths.**
4. **Reads TSV files and converts them into usable data.**
5. **Provides lookup methods for text and metadata.**

📦 VARIABLES (What they store and why)

1. cachedLanguageData

- Stores the language settings after loading them once.

- Prevents reloading the same asset multiple times.
- Acts like a **singleton cache**.

👉 Purpose: performance + consistency.

2. CultureCodeKey

- A fixed key used to save/load the selected language.
- Stored in persistent user preferences.

👉 Example:

- "en-US"
- "pt-BR"

3. assetsPath

- Stores the folder path where language files are located.
- Built from:
 - StreamingAssets path
 - A folder name defined in settings

👉 Prevents recalculating the path every time.

LANGUAGE SETTINGS

LoadLanguageSettings()

What it does:

- Loads the main configuration asset called "Language Data".
- Stores it in cachedLanguageData.

Important behavior:

- If already loaded → returns cached version.
- If missing → logs an error.

👉 This is the **entry point** of the entire system.

CULTURE (LANGUAGE) MANAGEMENT

GetSaveCultureCode()

What it does:

- Retrieves the currently selected language.

Logic:

1. Start with default language.
2. Check if user has a saved preference.
3. If yes → use saved value.
4. Update runtime setting.

👉 Ensures:

- The game always has a valid language.

SetSaveCultureCode(newCode)

What it does:

- Saves a new language choice.

Behavior:

- Updates runtime setting.
- Saves to persistent storage.
- Forces the system to remember it next time.

👉 This is what you call when the player changes language.

📁 FILE PATH MANAGEMENT

GetLanguageAssetsPath()

What it does:

- Builds the folder path where TSV files live.

Behavior:

- Uses StreamingAssets.
- Creates folder if it doesn't exist.
- Caches the result.

👉 Ensures:

- Files always have a valid location.

TABLE UTILITIES

RowAndColumnCounter(table)

What it does:

- Calculates:
 - Number of rows
 - Number of columns

Behavior:

- If table is empty → returns 0,0.

👉 Used to safely process TSV data.

AVAILABLE LANGUAGES

GetAvailableLanguages()

What it does:

Reads a TSV file (LanguageData.tsv) and builds a list of languages.

Expected table structure:

Row	Meaning
0	Culture codes
1	Native names
2	Display names
3	Availability (true/false)

Logic:

1. Load TSV file.
2. Loop through columns (languages).
3. Skip first columns (ID/labels).
4. Read:
 - culture
 - native name
 - display name
 - availability
5. Only include languages marked as available.

Output:

A list of language objects containing:

- culture code
- names
- column index

👉 This defines **what languages exist in your game**.

ID EXTRACTION

ExtractIDs(table, languages, cultureCode)

What it does:

Extracts localized text for a specific language.

Expected table structure:

Row	Meaning
0–3	Metadata
4+	Actual content

Logic:

1. Find column that matches the selected culture.
2. Loop through rows (starting at row 4).
3. Read:
 - ID (column 1)
 - Text (language column)
4. Convert ID to number.
5. Store as ID + text pair.

Output:

List of:

- ID
- localized text

👉 This is the **core translation data**.

📖 FULL DATA LOADING

GetAllData()

What it does:

Loads EVERYTHING needed for localization.

Files used:

- LanguageData.tsv → translations
- MetaData.tsv → text settings
- CanvasData.tsv → UI-related data

Process:

1. Validate files exist

If any are missing → stop.

2. Load all tables

3. Validate selected language

- If missing → fallback to default
- Notify system of change

4. Extract data:

- Text data → idData
- Canvas data → idCanvasData
- Metadata → parsed separately

5. Parse metadata JSON

Each metadata entry:

- Is stored as a JSON string in TSV
- Converted into structured metadata:
 - alignment
 - font size
 - font index
 - component type

If parsing fails:

- Logs warning
- Keeps minimal data

Final result:

The system now has:

- Translated text
- UI metadata
- Canvas data

👉 This is the **main loading pipeline**.

DATA RETRIEVAL

GetIDText(ids, id)

What it does:

- Finds text for a given ID.

Behavior:

- Returns text if found.
- Returns null if not found.

👉 Used everywhere in UI.

GetIDMeta(ids, id)

What it does:

- Retrieves metadata for a given ID.

Metadata includes:

- alignment
- font size
- font selection
- component type

👉 Used to **style UI dynamically**.

🔗 HOW EVERYTHING CONNECTS

Flow of the system:

1. **Load settings**
→ LoadLanguageSettings()
2. **Determine language**
→ GetSaveCultureCode()
3. **Find files**
→ GetLanguageAssetsPath()
4. **Read available languages**
→ GetAvailableLanguages()
5. **Load all data**
→ GetAllData()
6. **Use data**
→ GetIDText()
→ GetIDMeta()

🎯 USAGE (Practical Example)

When the game starts:

- Load settings
- Load saved language
- Load all data

When user changes language:

- Save new culture code
- Reload data
- Update UI

When UI needs text:

- Request text using ID

When UI needs styling:

- Request metadata using ID

IMPORTANT DESIGN NOTES

✓ **Strengths:**

- Data-driven (TSV + JSON)
- Flexible for multiple languages
- Supports UI customization
- Cached for performance

Constraints:

- Depends on strict TSV structure
- Requires valid IDs
- Metadata parsing can fail if JSON is invalid
- Language must exist in table

FINAL SUMMARY

This script is essentially:

A **localization engine** that loads structured data from files, keeps it cached, and provides fast access to translated text and UI settings based on IDs and selected language.

It separates:

- **Content (TSV files)**
- **Configuration (settings)**
- **Runtime usage (lookup methods)**

Which makes your system:

- Scalable
- Editable outside Unity
- Easy to expand for new languages or UI rules

This script is very small, but it plays a **critical architectural role** in your system.

It acts as a **global notification hub** for language changes — basically a way for different parts of your application to “talk” without being directly connected.

OVERALL IDEA

Think of this script like a **radio broadcast system**:

- Some system changes the language.
- It sends a signal: “Language updated!”
- Any object that cares (UI, text, menus, etc.) can listen and react.

👉 This avoids tight coupling between systems.

COMPONENTS EXPLAINED

1. The Class (LanguageManagerDelegate)

This is a **static container**.

What that means:

- It doesn’t need to be created or instantiated.
- It exists globally.
- Any part of your project can access it.

👉 It behaves like a **central communication point**.

2. The Delegate (LanguageUpdateDelegate)

What it is:

A **function signature definition**.

In simple terms:

It defines:

“Any method that wants to listen must look like this: no input, no output.”

Why it exists:

It ensures that all listeners follow the same structure.

👉 Think of it like a **plug standard**:

- Only compatible plugs can connect.

3. The Event (OnLanguageUpdate)

What it is:

A list of subscribed listeners.

Behavior:

- Other systems can register themselves.
- When triggered, all registered listeners are executed.

Important concept:

This is not just one function — it's a **collection of functions**.

👉 Example idea:

- UI Text system subscribes
- Menu system subscribes
- Dialogue system subscribes

All of them will react when the event fires.

4. The Method (NotifyLanguageUpdate)

What it does:

Triggers the event.

Behavior:

- Checks if there are listeners.
- If yes → calls all of them.

Important detail:

If nobody is listening → nothing happens.

👉 Safe to call anytime.

HOW IT WORKS TOGETHER

Step-by-step flow:

1. A system changes the language
(for example, user selects a new language)
2. That system calls:
→ `NotifyLanguageUpdate`
3. The event fires
4. All subscribed systems are notified:
 - UI text refreshes
 - Fonts may update
 - Layouts may change

USAGE (HOW YOU USE THIS SYSTEM)

1. Subscribing (Listening for changes)

A system that needs to react (like UI text):

- Registers itself to the event
- Provides a method to execute when triggered

👉 Example behavior:

- “When language changes, update my text.”

2. Unsubscribing (Important!)

When the object is no longer needed:

- It should remove itself from the event

👉 Prevents:

- Memory issues
- Unexpected behavior

3. Triggering the update

When language changes:

- Save new language
- Reload data
- Call notification

REAL USE CASE IN YOUR SYSTEM

This script connects directly with your **LanguageFileManager**.

Example flow:

1. Player changes language
2. Language is saved
3. Data is reloaded
4. NotifyLanguageUpdate is called
5. UI updates automatically

DESIGN ADVANTAGES

✓ Decoupling

Systems don't need to know about each other.

✓ Scalability

You can add new systems (like subtitles, audio, UI themes) without modifying existing code.

✓ Clean architecture

No need for manual updates everywhere.

POTENTIAL PITFALLS

Forgetting to unsubscribe

Can cause:

- Ghost updates
- Memory leaks

! Silent failures

If nothing is subscribed:

- Event does nothing
- No warning

🧠 FINAL SUMMARY

This script is:

A **global event dispatcher** that broadcasts language changes to any system that cares.

It turns your localization system from:

✗ Manual updates everywhere
into

✓ Automatic, reactive updates across the entire application

Rebuild Canvas

This script is an **Editor tool for reconstructing and saving UI canvases** using your localization system.

It connects three important things:

- JSON data (stored layouts)
- Scene objects (actual UI hierarchy)
- Editor tools (buttons, dropdowns, workflows)

👉 In short:

It lets you **load a saved UI layout into the scene, edit it visually, and save it back to data.**

🧠 OVERALL WORKFLOW

Think of it like a **round-trip system**:

1. Load saved canvas (JSON)
2. Rebuild it in the scene
3. Modify it visually
4. Save it back to JSON

MAIN COMPONENT (RebuildCanvas)

This is the component you attach to an object in the scene.

◆ VARIABLES

1. canvasObject

- The actual UI root object in the scene.
- This is where the hierarchy will be created.

👉 If you rebuild → this object gets replaced.

2. canvasStructure

- A structured representation of the canvas.
- Contains:
 - hierarchy paths
 - layout data
 - UI component settings

👉 This is the **data version of the UI**.

3. canvasID

- Identifies which canvas data entry you are working with.
- Links this object to saved data in the JSON file.

👉 Used for:

- selecting
- replacing
- tracking

CANVAS CREATION

CreateCanvasData(json)

What it does:

Builds a UI hierarchy in the scene from JSON data.

Step-by-step:

1. Registers undo (so you can revert in editor)
2. Validates JSON:
 - If empty → error
 - If invalid → error
3. Converts JSON → structured data
4. Stores that structure in canvasStructure
5. If an old canvas exists:
 - Deletes it (with undo support)
6. Creates a new canvas using:
 - hierarchy data
 - layout data
 - UI components

Result:

You get a **fully reconstructed UI layout in the scene.**

Important:

This uses the **CanvasManager system**, meaning:

- hierarchy is rebuilt exactly
- transforms are restored
- components are applied



CANVAS SAVING

SaveCanvasData()

What it does:

Extracts the current scene UI and converts it into JSON.

Step-by-step:

1. Registers undo
2. Reads the current canvas object
3. Extracts:
 - hierarchy
 - layout
 - components
4. Stores result in canvasStructure

5. Converts it into JSON

Result:

You get a **serialized version of the current UI**.

👉 Ready to:

- store
- version
- reuse

✂ EDITOR TOOL (Custom Inspector)

This is where things become powerful.

◆ VARIABLES (Editor)

fileData

- Path to JSON file storing all canvas data.

👉 Example:

- ProjectSettings/LanguageFileData.json

canvasSave

- List of all saved canvas entries.
- Each entry contains:
 - ID
 - context
 - JSON data

script

- Reference to the currently selected object.

🔧 INSPECTOR INTERFACE

OnInspectorGUI()

This builds the custom UI in the editor.

1. Load data

- Reads JSON file
- Populates canvasSave

2. Warning message

- Reminds user:
→ must load data from another tool first

3. Canvas selection dropdown

Shows options like:

- ID 1: Main Menu
- ID 2: Settings Menu

Behavior:

- Only works when one object is selected
- Disabled for multi-selection

4. Create button

"Create Canvas Hierarchy"

👉 When clicked:

- Calls CreateCanvasData
- Builds UI in scene

5. Save button

"Replace canvasID: X"

👉 When clicked:

1. Asks for confirmation
2. Extracts current canvas
3. Converts to JSON
4. Sends it to another editor window

Important:

This **overwrites existing data**

6. Extra messages

- Suggests enabling all objects for accurate capture
- Shows info/warnings

7. Default inspector

Shows:

- canvasObject
- canvasStructure
- canvasID

📁 JSON LOADING

LoadDataJson()

What it does:

Loads all canvas data from a JSON file.

Steps:

1. Check if file exists
2. Read file
3. Convert JSON → structured data
4. Extract canvas list
5. Store in canvasSave

Result:

Editor now has access to:

- all saved canvas layouts

HOW IT CONNECTS TO YOUR SYSTEM

This script depends on:

1. CanvasManager

- Builds and extracts UI hierarchies

2. LanguageFileManagerWindowData

- Stores all canvas entries

3. External editor tools

- Used to manage and store JSON

PRACTICAL USAGE

To LOAD a canvas

1. Select object with this component
2. Choose a canvas ID
3. Click "Create Canvas Hierarchy"

 Result:

UI appears in scene

To EDIT

1. Modify UI visually in scene
2. Move elements, resize, etc.

To SAVE

1. Click "Replace canvasID"
2. Confirm
3. Data is updated

IMPORTANT NOTES

✓ Advantages

- Visual editing of data-driven UI
- Reusable layouts
- Language-specific UI support
- Full undo support

Limitations

- Requires valid JSON structure
- Overwrites data (no versioning here)
- Depends on external tools being loaded first
- Works only inside editor

FINAL SUMMARY

This script is:

A **bridge between data and visuals**, allowing you to reconstruct UI layouts from JSON, edit them in the scene, and save them back into your localization system.

It transforms your workflow from:

 Hardcoded UI
into

 Data-driven, editable, reusable UI layouts

Component Converter

This script is an **Editor utility tool** designed to automatically convert standard UI and visual/audio components into versions that are compatible with your localization system ("LanguageTools").

Instead of manually replacing components one by one, this tool scans selected objects and attaches the correct "language-aware" components for you.

Overall Idea

Think of this script as a **batch converter**:

- You select objects in the Unity scene.
- You run the tool from a menu.
- It analyzes each object.
- It replaces or augments components with localization-ready versions.

It also:

- Prevents duplicates.
- Preserves undo functionality.
- Logs what happened.

Variables (State Control)

callCount

- Tracks how many times the conversion function has been triggered.
- This exists because Unity may call menu actions multiple times internally during multi-selection.
- It ensures the conversion logic runs **only once at the correct time**.

expectedCalls

- Stores how many objects are selected.
- Used together with callCount to delay execution until all internal calls are finished.

👉 Together, these two variables prevent:

- Running the conversion multiple times.
- Partial conversions when multiple objects are selected.

Entry Point (Main Trigger)

ConvertComponents()

This is the method triggered from the Unity menu:

GameObject → Tools → Language Tool → Converter to Language Tool

What it does:

1. Gets all selected objects.
2. Uses the call counter system to ensure proper timing.
3. Validates selection (shows error if nothing is selected).
4. Sends each object to be analyzed.

👉 This is the **starting point of the entire process**.

🔍 Object Analysis

`ObjectAnalysis(selectedObject)`

This is the **core decision-maker**.

What it does:

- Checks which components exist on the object.
- Based on what it finds, it calls the appropriate conversion method.

It supports:

- UI elements (Text, Button, Toggle, Image, etc.)
- TMP (TextMeshPro components)
- 3D text
- Audio
- Dropdowns and input fields

Important behavior:

- If a specialized component exists (like Button or InputField), it prioritizes that.
- If not, it falls back to simpler components (like Image or TMP text).

👉 This method is like a **routing system** that decides how each object should be converted.

🔄 Generic Text Conversion Helper

`ConvertTextComponent`

Used by:

- Button
- Toggle

What it does:

- Looks for text inside child objects.
- Supports:
 - Standard UI text
 - TextMeshPro text

If nothing is found:

- Logs an error.

👉 This avoids duplicating logic for components that contain text internally.

✂ Individual Converters (Core Work)

Each of these methods:

1. Checks if the object is already converted.
2. If yes → logs error and stops.
3. If not → adds the correct LanguageTool component.
4. Stores reference to the original component.
5. Registers Undo.
6. Logs success.

📄 Text Converters

ConvertText

- Converts standard UI text.
- Links it to a localization-aware text component.

ConvertTMPTText

- Same idea, but for TextMeshPro.

🖼 Image Converters

ConvertImage

- Converts UI images.

ConvertRawImage

- Converts raw images (usually textures or videos).

🕒 Interactive UI

ConvertButton

ConvertToggle

- These don't convert the button itself.
- Instead, they convert the **text inside the button/toggle**.

Dropdowns

ConvertDropdown

ConvertTMPDropdown

These are more complex:

- Add a helper component to control size.
- Convert dropdown into a localization-aware version.
- Extract all options (text + images).
- Store them in a new structure.

 This ensures dropdown options can also be translated.

Input Fields

ConvertInputField

ConvertTMPInputField

- Converts input fields so:
 - Placeholder text
 - User-facing strings can be localized.

Audio

ConvertAudioSource

- Converts audio sources into language-aware audio players.
- Useful for:
 - Voice localization
 - Language-based audio switching

3D / World Space

ConvertTextMesh

- Converts 3D text objects.

ConvertMeshRenderer

- Handles TMP text rendered in 3D with a mesh.

Safety Features

Every converter checks:

- If the object already has the converted version.
- If yes → stops and logs an error.

This prevents:

- Duplicate components
- Broken setups

Undo System

Every conversion:

- Registers with Unity's Undo system.

This means:

- You can press **Ctrl+Z**
- Everything will revert cleanly.

Usage (How to Use It)

Step-by-step:

1. Select one or more GameObjects in the scene.
2. Go to:
3. GameObject → Tools → Language Tool → Converter to Language Tool
4. The tool will:
 - Analyze each object.
 - Convert compatible components.
 - Log results in the console.

Practical Example

Before:

- Button with Text
- Image
- TMP Text

- Dropdown

After conversion:

- Button → text becomes localized
- Image → becomes language-aware image
- TMP Text → linked to localization system
- Dropdown → options become translatable

💡 Why This Script Is Useful

Without this tool:

- You'd manually replace every component.
- Risk of missing references.
- Very time-consuming.

With this tool:

- One click converts everything.
- Safe (undo + duplicate checks).
- Consistent setup.

🔗 Big Picture

This script is part of a larger system:

- **LanguageFileManager** → loads translations
- **Data structures** → store text & metadata
- **Delegate system** → notifies updates
- **This script** → prepares scene objects to use all that

👉 In short:

This is the **bridge between your scene objects and your localization system.**

Font Asset Bundle Builder

This script is an **Editor automation tool** focused on one specific job:

👉 turning font assets into **AssetBundles**, one file per font, with a consistent naming system.

It's designed for localization workflows where fonts may need to be:

- loaded dynamically,
- swapped per language,
- or distributed separately from the main build.

Overall Concept

Instead of manually creating bundles, this script:

1. Searches a specific folder.
2. Finds all valid font assets.
3. Builds a separate bundle for each one.
4. Applies a naming convention depending on the font type.
5. Logs success or failure.

👉 Think of it as a **batch processor for font packaging**.

Menu Command (Entry Point)

BuildFontAssetBundles

This is the only public method and acts as the **entry trigger**.

How it's used:

- You right-click in the Unity project window or use the menu:
- Assets → Tools → Language Tool → Build Font Asset Bundles

What it does step-by-step:

1. Locate the target folder

- Searches the project for a folder named "**AssetBundles**".
- Then filters to ensure it is specifically inside:
- Language Tool / AssetBundles

👉 This prevents accidentally using the wrong folder.

2. Validate the folder

- If the folder is not found:
 - Stops execution.
 - Logs an error.

3. Find assets inside the folder

It performs two separate searches:

Regular fonts

- Looks for standard font files (like system fonts).

TMP fonts

- Looks for TextMeshPro font assets.

👉 These are handled differently later.

4. Build bundles for each asset

For every asset found:

- Calls the internal build method.
- Passes:
 - the asset reference
 - the folder path
 - a flag indicating if it's a TMP font

5. Refresh the project

- Forces Unity to recognize newly created bundles.

⚙️ Core Build Logic

BuildFontAssetBundle

This is where the actual **bundle creation** happens.

Parameters (Inputs)

- **font identifier** → used to locate the asset
- **output path** → where the bundle will be created
- **is TMP flag** → determines how the asset is treated

Step-by-step behavior:

1. Resolve asset path

- Converts the internal identifier into a real file path.

- Extracts the file extension.

2. Validate file type

- If it's a normal font:
 - Only allows valid font formats (like common font files).
- If it's a TMP font:
 - Skips this validation (TMP assets are already valid).

👉 Prevents invalid files from being processed.

3. Generate bundle name

The name is based on:

- The original file name
- A custom extension:

Type	Bundle Extension
Regular font	.ltbundle
TMP font	.tmpltbundle

👉 This helps distinguish bundle types later.

4. Prepare build definition

- Creates a structure describing:
 - bundle name
 - which asset goes inside

👉 Each bundle contains **only one asset**.

5. Build the bundle

- Uses **chunk-based compression**
 - Better loading performance
 - Efficient memory usage
- Targets:
 - Windows standalone platform

6. Logging

- If successful:
 - Logs success message with bundle name.
- If something fails:
 - Catches the error.
 - Logs detailed failure info.

Validation Helper

IsValidFontFile

Purpose:

Checks if a file is a **valid font file**.

Accepted formats:

- TrueType fonts
- OpenType fonts
- Font collections

Behavior:

- Compares file extension against a known list.
- Ignores case differences.

👉 Ensures only real font files are processed.

Variables (What they store)

Inside main method:

- **guids**
 - Internal identifiers for assets/folders found in the project.
- **folderPath**
 - Final resolved path to the correct AssetBundles folder.

Inside build method:

- **assetPath**
 - Full path to the font file.
- **extension**
 - File type (used for validation).
- **bundleName**
 - Final name of the generated bundle.
- **build definition**

- Structure describing how the bundle should be built.

Usage (How to Use It)

Setup:

1. Create this folder structure:
2. Language Tool / AssetBundles
3. Place inside it:
 - Font files (like .ttf, .otf)
 - TMP font assets

Execution:

1. Select any asset or just open the menu.
2. Click:
3. Assets → Tools → Language Tool → Build Font Asset Bundles

Result:

- Each font becomes its own bundle.
- Files appear in the same folder.
- Named like:
 - myfont.ltbundle
 - mytmpfont.tmpltbundle

Why This Script Exists

In localization systems, fonts often vary by language:

- Japanese → different font
- Arabic → different font
- Cyrillic → different font

Instead of:

- Shipping all fonts inside the main build

You can:

- Load only the needed font at runtime using AssetBundles.

 This script automates that pipeline.

Limitations / Design Choices

- Builds only for **Windows platform**
- Assumes a **specific folder structure**
- One asset per bundle (simpler but more files)

Big Picture

This script fits into your system like this:

- **Localization data** → chooses language
- **Font system** → needs correct font
- **AssetBundles (this script)** → provides those fonts dynamically

 So this tool is essentially:

The **font packaging system** for your localization pipeline.

Language Data Auto Creator

This script is an **editor-side automation system** that guarantees your localization core asset (“Language Data”) always exists and is placed in the correct location—without forcing a rigid folder structure.

Think of it as a **self-healing setup tool**:

- If the asset exists → it reuses it.
- If it doesn't → it creates it.
- If the folder is missing → it builds the folder.
- If everything is missing → it falls back safely.

Overall Purpose

The script solves a very common problem in larger projects:

“Where is my central localization data stored, and how do I make sure it always exists?”

Instead of relying on manual setup, it:

- Finds the correct folder dynamically
- Ensures a **Resources** directory exists
- Creates or loads the localization data asset
- Optionally replaces it if needed
- Runs automatically on editor startup

Main System: LanguageDataAutoCreator

This is the **core manager** responsible for creation and retrieval.

Main Method

CreateOrGetData

This is the heart of the system.

What it receives:

- A flag that decides whether the user should be asked before replacing an existing asset.

Step-by-step behavior:

1. Locate the base folder

- Searches the entire project for a folder named:
- Language Tool
- This is done dynamically, meaning:
 - It doesn't matter where the folder is placed in your project.

2. Fallback safety

If the folder is not found:

- It logs a warning.
- Falls back to:
- Assets

 This ensures the system **never breaks**, even in incomplete setups.

3. Ensure a Resources folder exists

- Inside the base folder, it ensures there is a:
- Resources
- If it doesn't exist:
 - It creates it automatically.

 This is important because:

- Assets inside Resources can be loaded at runtime easily.

4. Define asset location

The final asset path becomes:

[BaseFolder]/Resources/Language Data

5. Try loading existing asset

- Checks if the asset already exists at that path.

6. If asset exists

Two possibilities:

✓ If overwrite is NOT allowed:

- Simply returns the existing asset.

⚠ If overwrite IS allowed:

- Prompts the user with a confirmation dialog:
 - Yes → deletes and recreates
 - No → keeps existing asset

7. Create new asset (if needed)

If no asset exists or overwrite is confirmed:

- Creates a new instance of the data container
- Saves it to the correct location
- Marks it as modified (so Unity tracks it)
- Forces Unity to save and refresh

8. Return result

- Always returns a valid asset at the end.

👉 This guarantees:

You always get a usable localization data object.

Manual Creation

CreateLanguageDataAsset

This method adds a menu option:

Assets → Create → Tools → Language Tool → Language Data

What it does:

1. Calls the main creation method with overwrite enabled.
2. Focuses the project window.
3. Selects the created asset automatically.

 This improves usability:

- You instantly see and can edit the asset.

Folder Search Utility

FindLanguageToolFolder

This is a helper method used internally.

What it does:

1. Searches all folders in the project.
2. Looks specifically for one named:
3. Language Tool
4. Ensures the name matches exactly (not partial).

Output:

- Returns the folder path if found
- Returns nothing if not found

 This makes the system:

- Flexible
- Independent of project structure

Automatic Initialization

LanguageDataStartup

This is a **startup hook**.

What it does:

- Runs automatically when the Unity Editor starts.

How it works:

- Registers a delayed call (so it runs after Unity finishes loading).
- Calls the creation system with overwrite disabled.

Result:

- If the asset exists → nothing changes
- If it doesn't → it gets created automatically

 This guarantees:

The localization system is always initialized.

Variables and Their Roles

Inside main logic:

- **baseFolderPath**
 - Stores where the “Language Tool” folder is located.
- **resourcesPath**
 - Points to where runtime-loadable assets should live.
- **assetPath**
 - Full path of the localization data asset.
- **existingAsset**
 - Holds the already existing asset (if any).

Usage

Automatic (no action needed)

- Open Unity.
- The system runs in the background.
- Ensures the asset exists.

 You don't need to do anything.

Manual (when you want control)

1. Go to:
2. Assets → Create → Tools → Language Tool → Language Data
3. Confirm overwrite if needed.
4. Asset is created and selected.

Why This Script Is Important

Without this system:

- Missing assets could break your localization system.
- Developers might forget setup steps.
- Folder structure could become inconsistent.

With this system:

- Everything is auto-managed.
- Setup is consistent across all machines.
- Errors are minimized.

Big Picture

This script acts as the **foundation initializer** of your localization system.

It ensures that:

- The core data asset exists
- It is always accessible
- It is always in a valid location

In Simple Terms

👉 This script is like a “**bootstrap system**”:

- It prepares the environment.
- It guarantees required data exists.
- It removes manual setup errors.

Language Prefab Creator

This script is essentially a **toolbox for quickly creating ready-to-use language system objects inside the Unity Editor**, especially UI elements and prefabs already configured for localization.

It removes manual setup work and ensures everything is created correctly and consistently.

🧠 Core Idea

Instead of manually:

- Creating UI elements (Canvas, Buttons, Text, etc.)
- Adding required components
- Setting up localization scripts

👉 This system lets you **create everything from the editor menu with one click**, already configured.

🔧 MAIN RESPONSIBILITY

The class acts as a **factory + setup assistant** that:

1. Finds the correct prefab
2. Instantiates it in the scene
3. Ensures required systems exist (Canvas, EventSystem)
4. Prepares the object for editing (undo, rename, unpack)

📁 VARIABLES

This script actually has **no persistent variables**.

Everything is:

- Static
- Executed on demand via menu actions

So instead of storing state, it **reacts to user actions**.

INTERNAL UTILITY METHODS

These are the “engine” behind the system.

CreateUICanvas()

What it does:

Creates a complete UI environment from scratch.

Step-by-step behavior:

- Creates a new object called **Canvas**
- Adds required UI systems:
 - Canvas (main UI container)
 - Scaler (handles resolution scaling)
 - Raycaster (detects clicks)
- Configures:
 - Render mode (screen overlay)
 - UI layer
 - Sorting order

Then:

- Creates an **Event System**
 - Handles input (mouse, keyboard, etc.)

Finally:

- Registers everything in the Undo system

Why this matters:

UI elements **cannot function without a Canvas and EventSystem.**

 This method guarantees they exist.

CreateAndConfigurePrefab(fileName, selectedObject, isUI)

What it does:

This is the **core creation method.**

It handles:

- Finding the prefab

- Deciding where to place it
- Creating it properly

Step-by-step:

1. **If it's UI:**
 - Try to find an existing Canvas
 - If none exists → create one automatically
2. **Find prefab**
 - Uses a helper method to locate it by name
 - If not found → logs error
3. **Determine parent**
 - If user selected something → use it as parent
 - If UI → attach to Canvas
 - Otherwise → root level
4. **Instantiate prefab**
 - Creates the object in the scene
5. **Finalize setup**
 - Calls another method to finish configuration

Why this matters:

This method ensures:

- Correct hierarchy
- No missing dependencies
- Clean setup every time

FinalizePrefabSetup(fileName, newGameObject)

What it does:

Prepares the object after creation.

Steps:

1. Registers Undo support
 - So user can undo creation
2. Unpacks prefab
 - Turns it into a normal editable object
 - No longer linked to prefab restrictions
3. Selects the object
 - So user can immediately interact with it

4. Triggers rename mode (F2)
→ Lets user rename instantly

Why this matters:

It improves workflow:

- Faster editing
- Immediate feedback
- No manual cleanup

MENU METHODS (USER ENTRY POINTS)

These are the **buttons you see in the Unity menu**.

Each one simply calls:

 CreateAndConfigurePrefab(...)

3D OBJECTS

These create non-UI objects.

Examples:

- Language Create File
- Language Script
- Audio Source
- Text (Legacy / TMP)

Usage:

Used for:

- World-space text
- Audio localization
- Script-driven language systems

UI OBJECTS (TMP)

These create modern UI elements using TextMeshPro.

Examples:

- Language Manager

- Dropdown
- Input Field
- Toggle
- Button
- Text

Behavior:

- Automatically ensures a Canvas exists
- Places UI correctly

Usage:

Used for:

- Menus
- UI text
- Language selection systems

UI LEGACY

Same idea, but for **older Unity UI system**.

Examples:

- Legacy Button
- Legacy Text
- Legacy Dropdown

SPECIAL CASE

Force Import of Fonts

This prefab:

- Likely forces font loading or bundling
- Used as a helper for localization systems

COMPLETE WORKFLOW

Here's how everything connects in real usage:

Developer Flow

1. Open Unity
2. Right-click in hierarchy
3. Navigate menu:
4. GameObject → Tools → Language Tool → UI → Button (TMP)
5. System:
 - Creates Canvas (if needed)
 - Instantiates prefab
 - Sets everything up
 - Selects object
 - Lets you rename instantly

Result

You get:

- Fully configured UI element
- Already compatible with localization system
- No manual setup needed

DESIGN INTENT

This system is designed to:

Save time

No repetitive setup

Prevent errors

No missing Canvas or components

Standardize structure

All objects follow same pattern

Improve workflow

Undo, auto-select, rename ready

IMPORTANT DETAILS

- Prefabs must exist in the project
- Naming must match exactly
- Works only inside the Editor (not in builds)

SIMPLE ANALOGY

Think of this script like:

👉 A “smart factory button”

Instead of building a car manually:

- You press a button
- The factory builds it
- Paints it
- Places it in your garage
- Hands you the keys

SUMMARY

This script is a **productivity tool for Unity Editor** that:

- Automatically creates UI and 3D language-ready objects
- Ensures required systems exist (Canvas, EventSystem)
- Instantiates prefabs correctly
- Prepares them for immediate editing
- Integrates smoothly with Undo and workflow

Language Settings Data Editor

This script is a **custom editor interface** that replaces the default inspector of your language settings asset with a much more powerful and organized tool.

Instead of showing raw fields, it builds a **structured control panel** where you can configure languages, fonts, file locations, and also *inspect runtime data*.

CORE PURPOSE

It transforms a simple data container into a **complete configuration dashboard** inside the Unity Editor.

It allows you to:

- Choose the default language from real system cultures
- Assign font databases
- Define where language files are stored
- Monitor what the system extracted at runtime

VARIABLES (FIELDS)

These variables exist only inside the editor and help build the UI.

availableCultures

- Stores **all languages supported by the system**
- Comes from the operating system's culture list

👉 Example: English, Portuguese, Japanese, etc.

availableCultureDisplayNames

- A list of **readable names** for those cultures
- Uses the **native language name**

👉 Example:

- "English"
- "Português"
- "日本語"

This is what appears in the dropdown.

currentSelectedCultureIndex

- Tracks which language is currently selected in the dropdown
- It's just a number pointing to a position in the list

UNITY LIFECYCLE METHODS

OnEnable()

What it does:

Runs automatically when the inspector is opened.

Behavior:

1. Retrieves all available system cultures
2. Creates a display list using their native names

Why:

So the dropdown shows **real languages instead of raw codes**

OnInspectorGUI()

What it does:

This is the **main function that draws the entire interface.**

Everything you see in the inspector is built here.

INSPECTOR SECTIONS

The interface is divided into clear sections.

1. Script Reference (Read-only)

What it shows:

- The script linked to the asset

Behavior:

- Disabled (cannot be edited)

Why:

Standard Unity practice for clarity

2. Archives Location

Variable used:

- folderName

What it does:

- Lets you define **where language files are stored**

Example:

StreamingAssets/MyLanguages

Why:

This is where your system loads localization data from

 3. Default Language Selection**Variables used:**

- availableCultures
- currentSelectedCultureIndex

What it does:

- Shows a dropdown of all system languages
- Lets you pick the default one

Behavior:

- Converts selection into a culture code (like "en-US")

Why:

This defines the **fallback language** for your game

 4. Font List Data (Regular)**Variable:**

- fontListData

What it does:

- Lets you assign a **font database**

Extra behavior:

- If assigned → expands and shows the font list

Why:

Different languages may need different fonts



5. Font List Data (TMP)

Variable:

- fontListDataTMP

Same idea as above, but for:

- TextMeshPro fonts



6. Canvas Log Reference

Variable:

- errorLanguageTool

What it does:

- Lets you assign a GameObject used for **error display**

Example use:

- Showing missing translation warnings in UI



7. Extracted Runtime Data (Read-only)

This is one of the most powerful parts.

Variables shown:

- selectedCulture
- availableLanguages
- idData
- idMetaData
- idCanvasData

What it does:

Displays **live data generated at runtime**

Important:

- Completely locked (read-only)

- Cannot be edited

Why this is useful:

You can **debug your localization system visually**:

- See which languages were loaded
- Check all IDs
- Verify metadata
- Inspect canvas mappings

👉 This avoids guessing what's happening internally

SAVE SYSTEM

At the bottom:

Behavior:

- If anything changes:
 - Marks asset as modified
 - Saves automatically

Why:

Ensures no data is lost

HOW EVERYTHING WORKS TOGETHER

Typical Workflow

1. Create or open Language Settings asset
2. Configure:
 - Folder path
 - Default language
 - Font lists
3. Run your game or load data
4. Return to inspector
5. See extracted data live

Debug Example

If something is wrong:

- Missing translation → check idData
- Wrong font → check font list
- Language not loaded → check availableLanguages

DESIGN GOALS

This editor was clearly built to:

Improve usability

No raw data editing

Prevent mistakes

Dropdown instead of manual typing

Centralize configuration

Everything in one place

Enable debugging

Live runtime data visibility

Integrate with Unity workflow

- Undo support
- Automatic saving
- Clean inspector layout

SIMPLE ANALOGY

Think of this like:

A control panel for your localization system

Instead of editing config files manually:

- You use switches, dropdowns, and panels
- You can see system status in real time

SUMMARY

This script creates a **custom inspector UI** that:

- Lets you configure language settings easily
- Provides a dropdown of real-world languages
- Manages font databases
- Defines file storage location
- Displays runtime localization data
- Automatically saves changes

Language Settings Window

This script creates a **Project Settings panel inside Unity** for managing your localization system in a centralized, clean, and efficient way.

Instead of hunting for a ScriptableObject in your project, this gives you a **dedicated configuration page**, like Unity's own settings (Graphics, Input, etc.).

CORE IDEA

It builds a **custom settings window** under:

Project Settings → Language Tool

From there, you can:

- Configure language folders
- Choose default language
- Assign fonts
- Set error UI
- Manage everything in one place

VARIABLE

cachedData

What it is:

A stored reference to your language settings asset.

Why it exists:

Without this, Unity would:

- Reload the asset every frame
- Waste performance
- Cause unnecessary refreshes

What it does:

- Loads once
- Reuses the same data

👉 This is a **performance optimization**

MAIN SYSTEM

CreateProvider()

What it does:

This method **registers the custom settings panel** in Unity.

It tells Unity:

“Create a new settings section called Language Tool.”

Important configuration:

- Path: "Project/Language Tool"
→ Where it appears in settings
- Scope: Project
→ Applies to the whole project, not just user preferences
- Label: "Language Tool"
→ Name shown in UI

GUI HANDLER (MAIN LOGIC)

This is the heart of the system.

It defines **everything you see and interact with**.

Step 1: Load Data (with caching)

- If no data is loaded → load it once
- Store it in `cachedData`

Step 2: Missing Asset Handling

If the settings asset doesn't exist:

What happens:

- Shows warning message
- Displays a button: **"Create Settings Asset"**

If clicked:

- Automatically creates the asset
- Selects it in the project
- Focuses it

Why:

Prevents the system from breaking

 Self-repair mechanism

DATA WRAPPING

Serialized representation

The script creates a **temporary editable version** of the data.

Why:

- Allows safe editing
- Supports undo
- Integrates with Unity's system

UI SECTIONS

1. Data Asset

What it shows:

- Reference to the settings asset

Extra buttons:

- **Ping**
 - Highlights asset in project
- **Open**
 - Opens it in inspector

Why:

Quick navigation and access

2. Archives Location

Variable:

- folderName

What it controls:

- Folder inside **StreamingAssets** where language files live

Features:

Text field

- Manually type folder name

Select button

- Opens system folder picker

Validation:

- Only accepts folders inside StreamingAssets
- If invalid → shows error popup

Why this matters:

Ensures localization files are always in the correct place

3. Default Language

What it does:

- Displays dropdown of all system languages

Behavior:

- Uses native names (user-friendly)
- Converts selection into culture code

Safety:

- Ensures valid index (no crashes)

Why:

Defines fallback language for the game

4. Font List Data (Legacy)

Variable:

- fontListData

What it does:

- Assigns font database for standard UI

Behavior:

- If assigned:
 - Shows full font list for editing
- If NOT assigned:
 - Shows error message

Why:

Missing fonts = broken UI text

5. Font List Data (TMP)

Variable:

- fontListDataTMP

Same idea, but for TextMeshPro.

Error handling:

If missing → warning shown

6. Error Canvas

Variable:

- errorLanguageTool

What it does:

- Reference to UI object used for error display

Example:

- Missing translation warnings
- Debug messages

CHANGE HANDLING

For each editable field:

Process:

1. Detect change
2. Register undo
3. Mark asset as modified
4. Save changes

Why:

- Supports Ctrl+Z
- Prevents data loss
- Keeps Unity in sync

KEYWORDS

This section adds search terms like:

- Language
- Localization
- Fonts
- TMP

Why:

So users can quickly find the settings via search bar

COMPLETE FLOW

User Workflow

1. Open:
2. Project Settings → Language Tool
3. Configure:
 - Language folder
 - Default language
 - Fonts
 - Error UI
4. System:
 - Saves automatically
 - Applies changes safely

Behind the scenes:

- Data is cached
- Changes are tracked
- Undo is supported
- Validation prevents mistakes

DESIGN GOALS

✅ Centralization

Everything in one place

✅ Performance

No repeated asset loading

✅ Safety

- Folder validation
- Null checks
- Error messages

✅ Usability

- Dropdowns instead of typing
- Folder picker
- Visual feedback

✅ Integration

Feels like native Unity settings

🧠 ANALOGY

Think of this as:

👉 A **control room panel for localization**

Instead of:

- Editing files manually
- Searching assets in folders

You:

- Open one panel
- Adjust everything from there

- See immediate results

SUMMARY

This script creates a **Project Settings panel** that:

- Loads and caches your language configuration
- Allows editing of localization settings
- Validates folder paths
- Provides system language selection
- Manages font databases
- Handles missing data safely
- Integrates with undo and saving systems

Language File Manager Window

This script is essentially a **complete editor tool for managing localization data** inside Unity. It acts like a mini “database editor” for languages, texts, UI canvases, and metadata—while also syncing everything with external files.

I'll break it down clearly: variables, methods, and how you actually use it.

Big Picture – What this tool does

This window allows you to:

- Create and edit localized text entries (with IDs).
- Manage UI canvas localization data.
- Detect duplicate IDs (very important for consistency).
- Load/save data to external files (TSV format).
- Switch between languages and edit each one.
- Keep everything synchronized between:
 - Editor memory
 - JSON cache (for editor state)
 - TSV files (final localization files)

Think of it as a **localization control center**.

Variables (Fields) Explained

Language & Data Storage

- **languageForEditing**
The current language being edited (like "en", "pt-BR").
Controls which column in the language files you're modifying.

- **componentSave**
A list of all text-based localization entries.
Each item contains:
 - ID
 - Text
 - Context
 - Font info
 - Alignment, etc.
- **canvasSave**
Stores UI canvas data (serialized as JSON).
Used for rebuilding UI layouts per language.
- **availableLanguages**
List of languages detected from the TSV files.
Includes:
 - Culture code
 - Display name
 - Whether it's enabled

UI State

- **scrollPosition**
Keeps track of scroll in the window.
- **firstTime**
Ensures initialization logic runs only once.
- **idIndex**
The currently selected ID in the editor.
- **showTextData / showCanvasData**
Toggles visibility of sections.
- **filesSaved**
Tracks if there are unsaved changes (used to highlight Save button).

File Paths

- **fileData**
Path to a JSON file that stores editor state.
- **assetsPath**
Root folder where language files live.
- **canvasDataFile / languageDataFile / metaDataFile**
Paths to the TSV files:
 - Canvas data
 - Text data
 - Metadata

Internal Helpers

- **duplicateID**
Stores duplicated text IDs.
- **canvasID**
Stores duplicated canvas IDs.
- **cultures / cultureNames**
All system languages for dropdown selection.
- **selectedCultureIndex**
Index of selected language.
- **trashImage**
Icon used for delete buttons.

Core Methods Explained

□ Window Lifecycle

ShowEditorWindow

- Opens the editor window from the menu.

OnEnable

Runs when the window opens:

- Initializes culture list.
- Loads icons.
- Sets file paths.
- Ensures TSV files exist (creates defaults if missing).

OnDestroy

- Saves current editor state to JSON.
- Unsubscribes from undo events.

OnGUI

This is the **main UI loop**:

- Detects duplicates.
- Initializes data (first time only).
- Draws:
 - Buttons
 - Language dropdown

- ID panel
- Component lists
- Handles scrolling and layout.

GUI Sections

DrawActionButtons

Provides main actions:

- **Open Folder** → Opens localization folder.
- **Save File** → Writes TSV files.
- **Load File** → Reads TSV files.
- **Rebuild Canvas** → Creates a helper object to rebuild UI.
- **Organize All Items** → Sorts everything.
- **Search by ID** → Focuses a specific entry.

DrawLanguageDropdown

- Lets you pick which language to edit.
- If language exists → asks to load it.
- Displays:
 - Current language
 - System language

DrawIdDisplayPanel

Handles ID navigation and validation:

If no duplicates:

- Shows min/max ID.
- Navigation buttons:
 - Jump
 - Increment/decrement
- Manual input field

If duplicates:

- Displays them in red.
- Allows deleting duplicates.

RenderComponentDisplay

Displays and edits a single localization entry:

Editable fields:

- ID
- Context
- Text
- Alignment
- Font size
- Font index

Also:

- Shows component type icon
- Allows deletion

Data Management Methods

CompareID

- Sorts both lists (text + canvas) by ID.

FindDuplicateIds

- Detects duplicate IDs in both datasets.

RemoveID

- Removes current ID.
- Optional confirmation dialog.
- Auto-adjusts selected ID afterward.

SaveLanguageFile

This is one of the most important methods.

What it does:

1. Sorts data.
2. Checks for duplicates (blocks save if found).

3. Confirms overwrite.
4. Loads existing TSV files.
5. Rebuilds headers (languages).
6. Converts internal data into rows.
7. Syncs:
 - Text data
 - Canvas data
 - Metadata
8. Writes everything back to TSV files.

LoadLanguageFile

What it does:

1. Validates files.
2. Loads TSV data.
3. Extracts:
 - Text entries
 - Canvas entries
 - Metadata
4. Rebuilds internal lists.
5. Merges metadata with text.

GetAllavailableLanguages

- Reads language columns from TSV.
- Builds list of available languages.

SaveDataJson / LoadDataJson

These manage **editor state only** (not localization data):

- Saves:
 - Current language
 - UI state
 - Lists
- Restores everything when reopening the window.

AddComponent / AddCanvas

Used by other systems:

- Adds or updates entries.
- Prevents duplicates.

- Keeps lists sorted.
- Marks data as unsaved.

How You Use This Tool

Basic Workflow

1. Open:
2. Tools → Language Tool → Language File Manager
3. Select a language.
4. Edit:
 - Text entries
 - Canvas data
5. Fix any duplicate IDs (if shown).
6. Click **Save File** → exports TSV files.

Typical Use Case

- You build UI → assign IDs to text.
- Use this tool to:
 - Fill translations
 - Adjust fonts/alignment
- Export → game reads TSV files at runtime.

Important Rules

- IDs must be unique → otherwise saving is blocked.
- Always save after changes.
- Canvas data is stored as JSON inside TSV.
- Metadata (font, alignment) is stored separately.

Key Design Strengths

- ✓ Separation of concerns:
 - Text
 - Canvas
 - Metadata
- ✓ Safe editing (Undo support)
- ✓ Duplicate detection
- ✓ Multi-language structure
- ✓ Editor state persistence
- ✓ External file compatibility (TSV)

Mental Model

Think of it like this:

Editor Window



Internal Lists (componentSave, canvasSave)



JSON (temporary state)



TSV Files (final localization data)



Game Runtime

Language ID Scanner Window

This script creates a **tool window inside the Unity Editor** that helps you **scan scenes for language IDs**, detect **duplicates**, and quickly **locate objects using those IDs**. It's essentially a debugging and validation tool for your localization system.

Core Idea (What this tool does)

It goes through all scenes in your project (specifically the ones in Build Settings), looks for any component that contains a field called “**iD**”, and records:

- Where that ID was found
- Which object contains it
- Whether that ID is duplicated

It then displays everything in a searchable, organized interface.

Variables (Fields Explained)

scrollPos

- Stores the current scroll position of the window.
- Ensures the UI remembers where you are when scrolling through results.

◆ sceneLanguageIDs

- A dictionary (key-value structure).
- **Key:** Scene path
- **Value:** List of all IDs found in that scene
- This is the main storage of scan results.

◆ currentScenePath

- Stores the scene you had open before running a scan.
- Used to restore your workspace after scanning finishes.

◆ sceneFoldoutStates

- Keeps track of which scenes are expanded or collapsed in the UI.
- Each scene has its own toggle state.

◆ searchID

- The ID typed by the user for targeted search.
- Used when you don't want to scan everything, just one specific ID.

🔑 Menu Access (How to open it)

ShowWindow()

- Adds a menu option:
Tools → Language Tool → Language ID Scanner
- Opens the window normally.

ShowWindowID(id)

- Opens the window and **pre-fills a specific ID**.
- Useful if another tool wants to jump directly into this scanner.

🖥️ User Interface (OnGUI)

This is the **main visual interface**.

It contains:

1. Scan Button

- Scans all scenes in Build Settings.
- Asks you to save changes first.
- Restores your original scene afterward.

2. Search by ID

- Input field for a specific ID.
- If the ID exists:
 - Enables search button
 - Scans all scenes only for that ID
- If invalid:
 - Field turns red
 - Button is disabled

3. Clear Results

- Clears all previous scan data.

4. Results Display

- Shows each scene as a collapsible section.
- Inside each scene:
 - Lists all found IDs
 - Highlights duplicates in red

Each result shows:

- ID value
- Component type
- Object name

5. Select Button

- Clicking it:
 - Opens the correct scene (if needed)
 - Selects the object in the hierarchy
 - Highlights it

Core Logic (How scanning works)

ScanAllScenesInBuild()

What it does:

1. Clears previous results
2. Gets all enabled scenes
3. Opens each scene one by one
4. Scans for IDs
5. Tracks how many times each ID appears
6. Marks duplicates

Important behavior:

- Uses a counter system to detect duplicates across scenes.

ScanSpecificID(id)

What it does:

- Similar to full scan, but:
 - Only keeps results that match the given ID
- Shows a popup with result count

ScanSceneForLanguageIDs(scene)

This is the **heart of the system**.

What it scans:

- Every object in the scene
- Every component attached to those objects

It looks for:

1. Fields named “**iD**”
2. That are integers

It also checks:

- Lists inside components

- If list items contain an “**ID**” **field**, it extracts those too

Output:

Creates entries with:

- ID value
- Component type
- Object name
- Full hierarchy path
- Duplicate flag (initially false)

GetHierarchyPath(object)

What it does:

- Builds a string like:

Canvas/Menu/Button/Text

Why:

- Allows the tool to **find the exact object later**
- Used when clicking "Select"



Helper Data Structure

LanguageIDInfo

Each found ID is stored as an object containing:

- **id** → The numeric ID
- **componentType** → Where it was found
- **objectName** → GameObject name
- **hierarchyPath** → Exact location in scene
- **isDuplicate** → Whether it appears multiple times



Usage (How you actually use this tool)



Typical workflow:

1. Open the tool:
 - Tools → Language Tool → Language ID Scanner
2. Click:

- **“Scan All Scenes”**
- 3. Review results:
 - Expand scenes
 - Look for red entries (duplicates)
- 4. Fix problems:
 - Click **Select**
 - Jump directly to the object
 - Correct the ID

Searching for a specific ID:

1. Enter ID in the field
2. Click **Search ID**
3. Jump between results quickly

Why this tool is important

Without something like this:

- Duplicate IDs can silently break localization
- You wouldn't know where IDs are used
- Debugging becomes painful

This tool gives you:

- ✓ Full visibility across all scenes
- ✓ Instant duplicate detection
- ✓ One-click navigation to problems

Design Strengths

- **Generic scanning** (doesn't depend on specific components)
- **Reflection-based** → finds any “iD” field automatically
- **Scene-safe workflow** (restores original scene)
- **Editor-integrated UX**

Simple Summary

This script is basically:

 **A search engine for localization IDs inside your entire project scenes**

It helps you:

- Find IDs
- Detect duplicates
- Navigate directly to problematic objects

Language Table Editor Window

This script creates a **custom editor window inside Unity** designed to manage and edit language tables stored in TSV (tab-separated values) files. Think of it as a **spreadsheet-like editor embedded directly in the engine**, with extra tools tailored for localization workflows.

I'll break it down clearly: first the **variables**, then the **methods**, and finally **how you actually use it**.

Overall Concept

The window:

- Loads TSV files (language tables).
- Displays them as a grid (rows × columns).
- Lets you edit values directly.
- Supports selection, filtering, resizing, and undo/redo.
- Saves changes back to file.

VARIABLES (What the script stores)

Core Data

tableData

- Holds the entire table.
- Each entry represents a row.
- Each row contains multiple columns (cells).

 This is the **main dataset being edited**.

currentFilePath

- Stores the path of the loaded file.

 Used when saving or reloading.

selectedRows / selectedColumns

- Lists of indices (numbers).
- Represent which rows/columns are selected.

👉 Used for filtering or focused editing.

showOnlySelected

- If enabled, only selected rows/columns are visible.

👉 Acts like a **filter mode**.

rowHeights / columnWidths

- Arrays storing visual sizes.

👉 Controls how big each row/column appears.

currentChargingMode

- Defines where the data came from.

Possible values:

- Custom file
- LanguageData
- MetaData
- CanvasData

👉 This determines **how saving behaves later**.

📁 Interaction State

These track what the user is doing:

isResizingRow / isResizingColumn

- True when dragging to resize.

initialResizePos

- Mouse position when resizing started.

originalSize

- Size before resizing began.

resizeIndex

- Which row/column is being resized.

mainScroll / tableScroll

- Scroll positions.

👉 One for the window, one for the table.

isDirty

- Indicates unsaved changes.

👉 If true → user modified something.

lastScroll

- Used to detect scroll movement.

👉 Prevents editing issues while scrolling.

File Paths

assetsPath

- Base folder for language files.

languageDataPath, metaDataPath, canvasDataPath

- Predefined file locations.

👉 Quick access buttons use these.

✏️ Constants

MIN_COLUMN_WIDTH, MIN_ROW_HEIGHT

- Prevents elements from becoming too small.

🎨 Colors

A large set of colors is defined to visually distinguish:

- Headers
- IDs
- Boolean values
- Editable vs non-editable cells
- Special rows

👉 This improves readability and meaning.

🎨 GUI Styles

Defines how text looks:

- Editable text
- Headers
- Labels
- Boolean dropdowns

👉 These control alignment, color, and font.

⚙️ METHODS (What the script does)

📖 Opening the Window

OpenWindow

- Opens the editor window from the menu.

👉 Entry point for the tool.

Lifecycle Methods

OnEnable

- Runs when window opens.
- Sets file paths.
- Registers undo/redo callback.

OnDestroy

- Runs when window closes.
- Saves temporary data.
- Unregisters callbacks.

OnUndoRedo

- Repaints UI after undo/redo.

GUI Rendering

OnGUI

Main loop that draws everything:

1. Initializes styles.
2. Shows scroll area.
3. If no data → show open buttons.
4. If data exists:
 - Show save options
 - Show visibility controls
 - Render table

👉 This runs every frame.

Style Setup

InitializeStyles

- Creates text styles once.
- Prevents repeated initialization.

File Handling

RenderOpenFileButtons

- Buttons to load files.

Options:

- Custom file
- LanguageData
- MetaData
- CanvasData

DrawOpenButton

- Helper for creating buttons.

LoadTableFrom

- Loads TSV file into memory.
- Initializes row/column sizes.

RenderSaveButtons

- Displays save options depending on mode.

SaveTableToFile

- Writes table back to disk.

AllowOverwrite

- Confirms before overwriting files.

DrawFixedSave

- Saves to predefined file paths.

DrawResetButton

- Clears everything (with confirmation).

ResetEditorState

- Fully resets:
 - Data
 - Selection
 - Scroll
 - Resize states

👁 Visibility & Selection

RenderVisibilityToggles

Controls:

- Show only selected rows/columns
- Manual input of indices
- Select all rows
- Select all columns
- Select columns where value is "True"

👉 This is a **power filtering system**.

Table Rendering

RenderTable

- Creates scrollable table area.
- Calculates total size.
- Calls sub-render methods.

RenderColumnHeaders

- Draws column labels.
- Handles resizing interaction.

RenderRowsAndCells

- Iterates rows.
- Draws:
 - Row labels
 - Cells
 - Resize handles

RenderRowCells

- Draws all cells in a row.

RenderRowLabel

- Draws row index on left.

IsRectVisible

- Optimization: only render visible cells.

Cell Rendering

RenderCell

- Decides how each cell behaves:
 - Editable text
 - Boolean dropdown
 - Label

RenderBoolCell

- Displays True/False dropdown.
- Updates value if changed.

RenderEditableTextCell

- Allows typing directly into cell.
- Updates data and marks as dirty.

GetCellColor

- Assigns meaning via color:
 - Headers
 - IDs
 - Booleans
 - Data rows

👉 Encodes structure visually.

ShouldCenterText

- Decides alignment.

Resizing System

HandleColumnResize

- Detects click on column edge.

RenderColumnResizeHandle

- Draws resize handle.

RenderRowResizeHandle

- Same idea for rows.

HandleResize

- Handles dragging:
 - Updates size dynamically
 - Stops when mouse released

HOW TO USE (Practical Workflow)

1. Open the tool

- Go to:
- Tools → Language Tool → Language Table Editor Window

2. Load a file

Choose one:

- Custom TSV
- LanguageData
- MetaData
- CanvasData

3. Edit the table

You can:

- Click cells and type
- Toggle True/False values
- Resize rows/columns
- Scroll like a spreadsheet

4. Filter data

- Select rows/columns manually
- Enable "Show only selected"

5. Manage selections

- Type indices manually
- Use buttons:
 - Select all rows
 - Select all columns
 - Select active ("True") columns

6. Save changes

- Save back to:
 - Original file
 - Or new custom file

7. Undo/Redo

- Fully supported via editor system.

Key Design Insights

- It behaves like a **lightweight spreadsheet editor inside Unity**.
- Uses **color coding** to give semantic meaning.
- Optimized with **visibility checks** (only draws what's visible).
- Built with **non-destructive editing** (undo + dirty state).
- Supports **structured localization workflows**.

Access Permission Checker

Here's a clear breakdown of how this script works, focusing on **concepts, variables, methods, and usage**, without referring to programming syntax.

Overall Purpose

This script is a **runtime safety checker**.

When your game starts, it verifies whether the game has permission to **read and write files** inside two critical folders:

- The **Assets folder** (main game data)
- The **StreamingAssets folder** (external/raw files often used for runtime loading)

If access is denied, it **shows a warning UI inside the game** so the user or developer knows something is wrong.

How the Script Flows

1. Game starts
2. Permission check runs automatically
3. It tries to write/delete a temporary file in key folders
4. If something fails → logs error + shows warning UI
5. If everything works → logs success

Variables Explained

◆ **No instance variables (important detail)**

This script mostly uses **temporary/local values**, not persistent fields.

However, during execution it uses:

hasAccessToAssets

- Stores whether the game can access the **Assets folder**
- Result of a permission test

hasAccessToStreaming

- Stores whether the game can access the **StreamingAssets folder**
- Also based on a permission test

testFile

- A temporary file name used to test permissions
- Created and deleted immediately
- Example purpose:
 - “Can I write here?”
 - “Can I delete files here?”

settings

- Holds configuration data loaded from your language system
- Contains a reference to a **warning UI object**

warning

- The actual **visual warning object** shown in the game
- Created dynamically if access fails

Methods Explained

InitializeCheckSettings

What it does:

- Runs **automatically when the game starts**
- Immediately triggers the permission check

Why it exists:

- Ensures the system validates access **before anything else depends on files**

CheckAccessPermissions

What it does:

- Checks both required folders:
 - Assets
 - StreamingAssets
- Calls the folder test method for each

Logic:

- If **either folder fails**:
 - Logs an error
 - Shows warning UI
- If both succeed:
 - Logs success message

Why it matters:

- Prevents silent failures later (like missing translations or data)

CheckFolderAccess(path)

What it does:

Tests if a folder is usable.

Step-by-step:

1. Checks if the folder exists
2. Creates a temporary file
3. Writes text into it
4. Deletes the file
5. Returns success or failure

What it detects:

- No permission to write
- No permission to delete
- Folder missing

- OS-level restrictions

Error handling:

- Logs detailed messages for:
 - File system errors
 - Permission denial

ShowWarning

What it does:

Displays a **visual warning inside the game**

Steps:

1. Loads language/settings data
2. Gets a predefined warning UI object
3. Creates that object in the scene
4. Marks it to **persist across scene changes**

Why it's important:

- Makes the issue visible even to non-developers
- Prevents silent failures

How Everything Connects

Game Starts



InitializeCheckSettings



CheckAccessPermissions



CheckFolderAccess (Assets)

CheckFolderAccess (StreamingAssets)



IF FAIL:

→ Log Error

→ ShowWarning (UI)

ELSE:

→ Log Success

Usage (How You Use This Script)

Automatic usage

You don't need to manually trigger anything.

Just having the script in your project means:

- It runs automatically at startup
- It protects your file-dependent systems

When this is useful

This script is especially important when:

- Running builds outside the editor
- Using external files (like language TSVs)
- Running on:
 - Mobile devices
 - Consoles
 - Restricted environments
 - Sandboxed systems

Problems it helps detect

- Game installed in a protected folder (like Program Files)
- Missing permissions on mobile devices
- StreamingAssets blocked or inaccessible
- Read-only environments

Design Insights

✓ Defensive programming

Instead of assuming access works, it **verifies first**

✓ Non-intrusive

- Doesn't crash the game
- Only shows a warning

✓ User-facing feedback

- Uses UI instead of just logs
- Helps even non-technical users understand the issue

✓ Clean test method

- Writing + deleting a file is the most reliable permission check

Key Takeaway

This script acts like a **gatekeeper**:

“Before the game relies on files, make sure the system actually allows file access.”

If not, it **warns clearly and immediately**, preventing hidden bugs later.

Language Initialization

Here's a **complete conceptual breakdown** of how this script works—covering **every variable, method, flow, and usage**, without referring to programming syntax.

Overall Purpose

This system is responsible for **initializing everything related to language at game startup**, including:

- Loading language settings
- Choosing the correct language
- Preparing font lists (both normal and TextMeshPro)
- Saving/loading font metadata to disk
- Loading fonts dynamically from external bundles

Think of it as the **entry point of your localization system**.

High-Level Flow

When the game starts:

Initialize Language System



Load language settings



Choose correct language (saved or system-based)



(Outside editor only)



Prepare font folder



Save font lists to files



Load fonts from asset bundles



Reload font lists from files (validation step)

Variables Explained

settingsData

- Holds **all language configuration**
- Includes:
 - Available languages
 - Default language
 - Selected language
 - Font list references

 This is the **core data container** of the system.

◆ fontListData

- Stores a list of **regular fonts**
- Used by legacy text systems

👉 Example:

- Arial
- Custom UI fonts

◆ fontListDataTMP

- Stores **TextMeshPro fonts**
- Used for modern UI text rendering

👉 TMP fonts are different because they:

- Support advanced text rendering
- Are required for TextMeshPro components

◆ folderPath

- Path to a folder where:
 - Font lists are saved
 - Asset bundles are stored

👉 Typically:

GameData/FontData

🚀 Initialization Method

InitializeLanguageSettings

What it does:

This is the **main entry point**.

It runs automatically when the game starts.

Step-by-step:

1. Defines where font data will be stored
2. Logs startup message
3. Loads language settings
4. Chooses the correct language

Then (only outside the editor):

5. Creates font folder if missing
6. Saves font lists to files
7. Loads fonts from bundles:
 - Regular fonts
 - TMP fonts
8. Reloads font lists from files (cleanup/validation)

Why this separation matters:

- Editor → fast iteration
- Build → real file system + bundles

Language Setup Methods

LoadLanguageSettings

What it does:

- Loads language configuration from a central manager
- Extracts:
 - Regular font list
 - TMP font list

If something fails:

- Logs an error
- Stops initialization early

SetupDefaultLanguage

What it does:

Chooses which language the game should use.

Step-by-step:

1. Gets the system language
 - Example: "en-US", "pt-BR"
2. Loads available languages
3. Checks if user already selected a language:
 - If yes → use saved one
 - If no → continue
4. Checks if system language is supported:
 - If yes → use system language
 - If no → use default language
5. Saves the chosen language
6. Loads all language-related data into memory

Why this is smart:

- Respects user preference first
- Falls back gracefully
- Avoids unsupported languages



Font Loading (Asset Bundles)

LoadFontsFromAssetBundleLegacy

What it does:

Loads **regular fonts** from external files.

Process:

1. Finds all files with a specific extension
2. For each file:
 - Opens it
 - Reads contained assets
 - Extracts fonts

- Adds them to the font list

Error handling:

- Logs if bundle fails to load
- Logs if font list is missing

LoadFontsFromAssetBundleTMP

What it does:

Same as above—but for **TextMeshPro** fonts

Key difference:

- Uses TMP font format instead of regular fonts

Why both systems exist:

- Some projects still use legacy text
- TMP is more modern and powerful



Font List File System

SaveFontListsToFile

What it does:

Saves **font names** into text files.

Important detail:

It does NOT save the fonts themselves
→ only their names

Example file content:

Arial

Roboto

CustomFont01

Why this exists:

- Keeps a lightweight reference list
- Helps validate which fonts should be used
- Avoids loading unnecessary fonts

LoadFontListsFromFile

What it does:

Reads previously saved font names and **filters the font lists**

Step-by-step:

1. Reads text files
2. Extracts font names
3. Compares with loaded fonts
4. Keeps only matching fonts

Result:

- Removes unwanted fonts
- Ensures consistency
- Prevents mismatches between builds

How Everything Connects

Language Settings



Font Lists (from settings)



Font Bundles (external)



Font Files (saved names)



Final Validated Font Lists

Usage (How You Use This System)

Automatic usage

You don't need to manually call anything.

Just having this system in your project:

- Initializes language automatically
- Loads correct fonts
- Ensures consistency

When this is useful

This system shines when:

- Supporting multiple languages
- Using dynamic fonts
- Shipping fonts separately (DLC/modding)
- Reducing build size via asset bundles

Problems it solves

- Missing fonts in builds
- Incorrect language selection
- Font duplication
- Unsupported system languages

- Large memory usage from unnecessary fonts

Design Insights

✓ Modular architecture

- Language settings
 - Font loading
 - File system
- All separated cleanly

✓ Runtime flexibility

- Fonts can be added externally
- No need to rebuild the game

✓ Validation layer

- Saves → reloads → filters
- Ensures data integrity

✓ Platform-aware behavior

- Editor vs Build handled differently

✓ Future-proof

- Supports both legacy and TMP systems

Key Takeaway

This script is essentially your **language boot system**:

It decides *what language to use*, *what fonts are available*, and *ensures everything is loaded correctly and efficiently* before the game actually starts using text.

Here's a **full conceptual explanation** of this script—covering **every variable, method, and how to use it**, without relying on programming syntax.

Overall Purpose

This script is a **localized audio system**.

It automatically:

- Loads an audio file based on the **current language**
- Updates when the language changes
- Optionally plays the audio through an audio component

Core Idea

Instead of manually assigning audio clips per language, this system:

Language → Folder → Audio File → Load → Play

So if the language changes from English to Portuguese, the audio automatically switches to the correct file.

How the System Flows

Component enabled



Subscribe to language change event



Load language settings



Build file path (based on selected language)



Check if file exists



Load audio asynchronously



Play audio (if enabled)

Variables Explained

◆ Inspector Variables (Editable in Editor)

These are the ones you control in the Unity Inspector.

fileName

- The name of the audio file to load
- Must include extension

👉 Example:

Hello.wav

VoiceLine_01.mp3

audioType

- Defines the **format of the file**

👉 Examples:

- WAV
- MP3
- OGG

⚠ Must match the actual file type

useAudioSource

- Determines if the system should **play the audio automatically**
- true → plays sound
- false → only loads it

audioSource

- Reference to the component that plays sound

👉 If useAudioSource is enabled, this must exist

audioClip

- Stores the **loaded audio file in memory**

👉 Automatically updated when a new file is loaded

🔒 Private Variables (Internal Logic)

languageData

- Holds current language configuration
- Includes selected language (culture)

previousFilePath

- Stores the last loaded file path

👉 Prevents reloading the same file repeatedly

filePath

- The **final resolved path** to the audio file

👉 Built dynamically based on language

🔄 Unity Lifecycle Methods

OnEnable

What it does:

- Activates when the object becomes active

Actions:

1. Subscribes to a **global language change event**
2. Immediately triggers an update

Why:

- Ensures audio is always correct when object appears
- Handles cases where language was already set

OnDisable

What it does:

- Runs when object is disabled

Action:

- Unsubscribes from language event

Why:

- Prevents memory leaks
- Avoids unnecessary updates



LanguageUpdate

This is the main method

What it does:

- Loads language settings
- Resolves correct audio file
- Starts loading process

Step-by-step:

1. Validate Audio Source

If audio playback is enabled but no audio component exists:

- Logs error
- Stops execution

2. Load Language Data

Gets current language settings

If it fails:

- Logs error
- Stops execution

3. Build File Path

Constructs a path like:

GameFolder/LanguageAssets/Assets/en-US/Sounds/Hello.wav

👉 Structure:

BasePath / Assets / Culture / Sounds / FileName

4. Avoid Duplicate Loading

If the file path is the same as before:

- Do nothing

👉 Optimization to avoid unnecessary work

5. Check File Exists

If file is missing:

- Logs error with path + language
- Stops execution

6. Clean Previous Audio

If an audio file is already loaded:

- Removes it from memory

👉 Prevents memory leaks

7. Start Loading

Begins asynchronous loading process

⌚ Asynchronous Loading

LoadFileCoroutine

What it does:

Loads the audio file **without freezing the game**

Step-by-step:

1. Create Request

Prepares a request to load the file from disk

2. Wait for Completion

Game continues running while loading happens

3. Check for Errors

If something fails:

- Logs error
- Stops

4. Extract Audio

Converts downloaded data into usable audio

5. Play Audio (Optional)

If:

- Audio exists
- AudioSource is enabled

Then:

- Assign clip
- Play sound

How Everything Connects

Language Change Event



LanguageUpdate



Resolve File Path



LoadFileCoroutine



AudioClip Created



AudioSource Plays Sound

Usage (How You Use It)

Basic Setup

1. Add this component to a GameObject
2. Set:
 - File name (e.g., Hello.wav)
 - Audio type (WAV/MP3/etc.)
3. Assign an AudioSource (optional but recommended)

Required Folder Structure

Your files must follow this pattern:

LanguageAssets/

└─ Assets/

│ └─ en-US/

│ └─ Sounds/

│ └─ Hello.wav

│ └─ pt-BR/

│ └─ Sounds/

│ └─ Hello.wav

Automatic Behavior

- When the game starts → audio loads
- When language changes → audio updates
- When object is enabled → audio refreshes

Example Use Cases

Voice Lines

- Character dialogue per language

UI Sounds

- Buttons with localized voice feedback

Tutorials

- Spoken instructions per region

Narration

- Storytelling in multiple languages

Common Pitfalls

Wrong file name

- Must match exactly (case-sensitive in some systems)

Wrong format

- File type must match selected audio type

Missing AudioSource

- Required if playback is enabled

❌ Missing files per language

- Every language must have the file

💡 Design Insights

✓ Event-driven system

- Reacts automatically to language changes

✓ Lazy loading

- Loads only when needed

✓ Memory safe

- Cleans previous audio before loading new

✓ Scalable

- Works with any number of languages

✓ Non-blocking

- Uses asynchronous loading

🧠 Key Takeaway

This script is essentially:

A **smart audio loader** that automatically picks and plays the correct sound file based on the current language, without requiring manual setup per language.

Language Create File

This script is basically an **automatic localized file generator**.

It listens for language changes, translates predefined lines if needed, and writes everything into a file on disk—either in the Unity Editor or in a built game.

Let's break it down clearly: variables, methods, and how you actually use it.

Core Idea

The script:

1. Loads the current language settings.
2. Goes through a list of lines.
3. Replaces some lines with translated text (based on IDs).
4. Writes all lines into a file.

It reacts automatically when the language changes.

Variables (Fields)

◆ File configuration

- **fileName**
The name of the file that will be created.
Example: "Test File" → results in Test File.txt.
- **fileExtension**
The file type.
Example: .txt, .json, .csv.

◆ Folder paths

- **folderInUnity** (*Editor only*)
Where the file will be saved when running inside the editor.
Example: "Editor" → saved inside the project Assets folder.
- **folderInBuild**
Where the file will be saved in the final game build.
Usually something like "StreamingAssets".

◆ fileLines (MOST IMPORTANT)

This is the **content of your file**.

Each item contains:

- **text** → what will be written

- **iD** → used to fetch translated text
- **translateText** → whether to replace the text with a translation

Example concept:

- A line can be static → just write text.
- Or dynamic → replace with translation using ID.

◆ languageData

- Stores the current language configuration.
- Contains:
 - selected language
 - translation database
 - IDs and texts

Unity Events

◆ OnEnable

- Subscribes to a global language update event.
- Immediately triggers an update.

👉 Meaning:

As soon as the object becomes active → the file is generated.

◆ OnDisable

- Unsubscribes from the event.

👉 Prevents errors or memory leaks.

Core Method

◆ LanguageUpdate()

This is the **heart of the script**.

What it does step-by-step:

1. **Loads language settings**
 - If it fails → stops.
2. **Loops through all lines**

- For each line:
 - If translation is enabled:
 - Uses the ID to find the translated text.
 - Replaces the original text.
- 3. **Creates the file**
 - Calls the method that writes everything to disk.

Helper Methods

GetFolderPath()

Decides where to save the file depending on environment:

- In Editor → uses project folder.
- In Build → uses streaming assets folder.

 This makes the script work both in development and in the final game.

CreateFile(folderPath)

Handles the actual file writing.

Steps:

1. Ensures the folder exists.
2. Builds the full file path.
3. Writes each line into the file.
4. Logs where the file was created.

Editor Tools (Extra Power)

These only exist inside Unity Editor.

Custom Inspector

Adds:

- A button to import language IDs.
- A preview of the file content.

◆ Import Settings Button

- Opens tools to manage translations.
- Helps assign IDs and texts.
- Prevents duplicate IDs unless confirmed.

◆ File Preview

Shows:

- Final file name
- All lines exactly as they will be written

👉 Very useful to debug before generating.

◆ Custom Property Drawer

Controls how each line appears in the inspector:

- If **translateText = true**
→ shows ID field.
- If **false**
→ hides ID field.

👉 Keeps the UI clean and intuitive.

🚀 Usage (How you actually use this)

Step 1 — Add the component

Attach it to any object in your scene.

Step 2 — Configure file

- Set file name
- Set extension
- Choose folder

Step 3 — Define lines

Example concept:

- Line 1 → static text
- Line 2 → translated using ID
- Line 3 → static again

Step 4 — Run the game (or trigger language change)

The script will:

- Translate lines
- Generate the file automatically

Practical Example

Imagine you're generating a **localized dialogue file**:

- Line 1 → "Game Start"
- Line 2 → ID = 10 (translated to "Iniciar Jogo" in Portuguese)
- Line 3 → "Version 1.0"

Result:

Game Start

Iniciar Jogo

Version 1.0

Important Details

- If translation is missing → original text stays.
- File is overwritten every time.
- Runs automatically on language change.
- Works both in Editor and Build.

Summary

This script is a **bridge between your localization system and external files**.

It:

- Listens for language changes
- Converts IDs into translated text
- Writes everything into a real file

👉 Think of it as:

“Export localized content into a file automatically.”

Language Script

This script is the **core runtime bridge between your language system and your scene objects**.

It takes translation IDs, fetches the correct text for the current language, and **pushes that text into UI or 3D elements using events**.

Think of it as:

“Take an ID → get translated text → send it to whatever needs to display it.”

🧠 Core Concept

Instead of directly assigning text to UI elements, this script:

1. Stores **IDs** instead of fixed text
2. Fetches the correct translation at runtime
3. Sends the result through **events** (very flexible)

📦 Variables (Fields)

◆ debug

- Enables debug messages in the console.
- When active, every update prints:
 - the ID
 - the translated result

👉 Useful for checking if translations are working.

◆ scriptTexts (VERY IMPORTANT)

This is the **main data of the script**.

It's a list where each entry contains:

- **ID** → identifier used to fetch translation
- **text** → the resolved (translated) text
- **targetScripts** → a set of events that receive the text

👉 Each entry = one localized element in your scene.

◆ languageData

- Stores the current language configuration.
- Loaded at runtime.
- Contains:
 - selected language
 - all translation entries (IDs → text)

🔄 Unity Events

◆ OnEnable

- Subscribes to a global language update event.
- Immediately updates all texts.

👉 This ensures:

- Text is correct when the object appears
- Text updates when language changes

◆ OnDisable

- Unsubscribes from the event.

👉 Prevents:

- errors
- memory leaks
- unwanted calls

⚙️ Core Method

◆ LanguageUpdate()

This is the **heart of the system**.

Step-by-step:

1. Load language data

- If it fails → stops everything

2. Loop through all entries

For each item in the list:

a) Get translation

- Uses the ID to find the correct text

b) Store result

- Updates the entry's text value

c) Debug (optional)

- Prints the result if debug is enabled

d) Apply result

- Sends the text to all connected targets

Utility Method

◆ ApplyUnityEvent(event, value)

This method is what makes the system **flexible and powerful**.

What it does:

1. Looks at all connected listeners (targets)
2. For each one:
 - Checks if it has a valid method
 - Ensures the method accepts a string
3. Calls the method and passes the translated text

Why this is powerful

You are NOT limited to UI text.

You can send the translated string to:

- UI text components
- 3D text meshes
- dialogue systems
- debug logs
- animations

- anything that accepts a string

Editor Features

◆ Custom Inspector

Adds an **Import Settings** button.

◆ Import Settings Button

- Checks if IDs already exist
- Asks before overwriting
- Opens an editor window for each entry

👉 Helps manage translations directly from the inspector.

Usage (How to use it)

Step 1 — Add the component

Attach it to any GameObject.

Step 2 — Add entries

For each text you want:

- Assign an **ID**
- Optionally add default text
- Connect a **target event**

Step 3 — Connect targets

Example:

- Connect to a UI Text component
- Select a method like “SetText”

Now when the script runs:

- It will send the translated text to that method

Step 4 — Run the game

The script will:

- Load language
- Replace all texts
- Update all connected objects

Practical Example

Imagine:

You have:

- A UI button
- A text component
- ID = 10 → "Play" (English), "Jogar" (Portuguese)

Flow:

1. Script runs
2. Gets ID 10
3. Finds "Jogar"
4. Sends it to the button text

Result:

 Button shows **"Jogar"**

Important Details

- If ID is invalid → result may be empty
- If no event is connected → nothing happens
- Works automatically when language changes
- Supports multiple targets per entry

Design Philosophy

This script is built around **decoupling**:

Instead of:

- "This script controls UI"

It does:

- “This script provides text → others decide what to do with it”

Summary

This script:

- Uses IDs instead of hardcoded text
- Fetches translations dynamically
- Sends results through events
- Updates automatically on language change

👉 In short:

It's a **universal localization dispatcher for your scene.**

Language Text Mesh / Language Text Mesh TMP

This script is a **specialized localization component for TextMeshPro.**

Its job is simple but powerful:

It automatically updates **text, font, and font size** of a TextMeshPro element based on the current language.

Core Idea

Instead of manually changing UI text for each language, this script:

1. Uses an **ID** to identify the text
2. Fetches the correct translation
3. Applies:
 - the translated text
 - the correct font
 - the correct font size

👉 All automatically when the language changes.

Variables (Fields)

◆ textComponent

- Reference to the TextMeshPro element in the scene.
- This is the object that will be modified.

👉 If this is missing → nothing works.

◆ **translateText**

- Boolean flag (true/false).
- Controls whether translation should happen.

👉 Useful when:

- You want to control text manually
- Or only use font/size adjustments

◆ **iD**

- Unique identifier for this text.
- Used to find:
 - translated text
 - font settings
 - size settings

👉 This connects the object to your language database.

◆ **languageData**

- Stores the currently loaded language configuration.
- Contains:
 - all translations
 - metadata (font, size, etc.)

🔄 **Unity Events**

◆ **OnEnable**

- Subscribes to a global language update event.
- Immediately applies localization.

👉 Ensures:

- Correct text on startup
- Updates when language changes

◆ OnDisable

- Unsubscribes from the event.

👉 Prevents unwanted updates or errors.

⚙️ Core Method

◆ LanguageUpdate()

This is where everything happens.

Step 1 — Validate component

- Checks if the TextMeshPro reference exists
- If not → logs error and stops

Step 2 — Load language data

- Retrieves current language configuration
- If it fails → stops execution

Step 3 — Apply translated text

Only if **translateText** is enabled:

1. Looks up the ID in the language database
2. Gets the translated string
3. Applies it to the text component

👉 If no translation is found → keeps current text

Step 4 — Apply metadata (VERY IMPORTANT)

This script doesn't just change text — it also adapts presentation.

It retrieves metadata using the same ID:

◆ Font Size

- If a valid size exists → apply it

👉 Useful because:

- Different languages may need different sizes
- Example: German text is longer than English

◆ Font

- Gets a font from a predefined list using an index
- Applies it to the text component

👉 Critical for:

- Asian languages
- Cyrillic
- Arabic
- Any special character set

✂ What is “Metadata”?

For each ID, you can store extra info like:

- font size
- font type

So one ID controls:

- content (text)
- appearance (font + size)

✂ Editor Features

◆ Custom Inspector

Adds an **Import Settings** button.

◆ Import Settings Button

This does something very useful:

1. Reads current TextMeshPro settings:
 - text

- font size
- font
- 2. Converts them into language data
- 3. Opens an editor window to save them

Why this matters

Instead of manually typing everything:

- You design your UI visually
- Click import
- It becomes localization data

◆ Safety Check

If the ID already exists:

- It asks before overwriting

👉 Prevents accidental data loss

🚀 Usage (How to use it)

Step 1 — Add the component

Attach it to a GameObject with a TextMeshPro element.

Step 2 — Assign TextMeshPro

- Drag your text component into the field

Step 3 — Set ID

- Choose an ID that exists in your language system

Step 4 — Enable translation

- Keep `translateText` enabled (usually)

Step 5 — Run the game

The script will:

- Load the correct language
- Update text
- Adjust font
- Adjust size

Practical Example

You have:

A UI label with ID = 20

Language data:

Language	Text	Font	Size
English	"Start"	Arial	36
Japanese	"スタート"	JP Font	42

What happens:

- Player switches language to Japanese
- Script runs

 **Result:**

- Text → "スタート"
- Font → Japanese font
- Size → 42

Important Details

- If ID is invalid → nothing changes
- If font index is wrong → font may not update
- If translateText is disabled → only font/size apply
- Runs automatically on language change

Design Philosophy

This script combines:

1. Content

- The actual translated text

2. Presentation

- Font
- Size

👉 This is important because localization is not just translation — it's also **visual adaptation**.

Summary

This script:

- Links a TextMeshPro element to a language ID
- Updates text automatically
- Adjusts font and size per language
- Reacts to language changes in real-time

👉 In short:

It's a **smart localized TMPro controller**.

Language Font List Data / Language Font List Data TMP

This script is one of the simplest pieces in your system, but it plays an important structural role: it acts as a **container for fonts used in localization**, specifically for TMPro.

Overall Purpose

The script defines a **data asset** that stores a list of TMPro font assets. Instead of hardcoding fonts in multiple places, everything is centralized in one reusable asset.

This allows your localization system to:

- Switch fonts depending on language (e.g., Latin → Japanese → Arabic).
- Keep font references organized and editable.
- Support dynamic font assignment at runtime.

Main Data Structure

◆ TMPFontList

This is the **only variable in the runtime part of the script**.

What it is:

A list (collection) that stores multiple TextMeshPro font assets.

What it represents:

Each entry is a font that your localization system can use.

Why it exists:

Different languages often require different fonts:

- English → standard Latin font
- Japanese → font with kanji support
- Arabic → font with RTL support

Instead of swapping fonts manually everywhere, your system can:

- Store all fonts here
- Reference them by index or configuration

📦 Script Type (Important Concept)

This is not a scene component. It is a **data asset**.

What that means:

- It exists as a file in your project (like a configuration file).
- You create it from a menu.
- It does not “run” like a normal behavior.

Practical effect:

Other systems (like your language system) **read from this asset** when needed.

⚙️ Editor Behavior (Custom Inspector)

The second part of the script only works inside the editor and improves usability.

🟡 Drag-and-Drop Area

What it does:

Creates a visual box where you can drag font assets into it.

How it behaves:

- When you drag font files into the area:
 - They are automatically added to the list.
 - Duplicates are ignored.
- It visually indicates that you can drop items there.

Why this is useful:

Without this, you would:

- Manually add list entries
- Assign each font one by one

With this:

- You can drop multiple fonts at once
- Faster workflow
- Cleaner UX

Duplicate Prevention

When adding fonts:

- The script checks if the font already exists in the list
- If it does → it skips it

This prevents:

- Redundant entries
- Index inconsistencies

Undo Support

The editor records changes so you can:

- Undo font additions

Marking Changes

After modifying the list:

- The asset is flagged as “modified”

This ensures:

- Changes are saved properly

How It Fits Into Your System

This script doesn’t do anything alone — it’s used by other parts of your localization system.

Example flow:

1. You create this asset and add fonts to it
2. Your language system loads language settings
3. That settings object references this font list
4. Another script (like your TMP text handler) asks:
 - “Which font should I use for this language?”
5. It retrieves the correct font from this list

Real Usage Example

Scenario:

You support:

- English
- Japanese
- Russian

You would:

- Add 3 fonts to this list:
 - Font A (Latin)
 - Font B (Japanese)
 - Font C (Cyrillic)

Then:

Your localization system decides:

- English → use index 0
- Japanese → use index 1
- Russian → use index 2

Important Limitations

- This script **does not decide which font to use**
- It **only stores fonts**

The logic for choosing fonts exists elsewhere (like your metadata system).

When You Should Use This

Use this asset when:

- You have multiple languages
- Different languages require different fonts
- You want centralized font management

Mental Model

Think of this script as:

A library shelf of fonts

Other systems walk in, pick the correct book (font), and use it.

Language Settings Data

This script is the **central configuration asset of your entire localization system**. Everything else you showed earlier (audio, text, fonts, file creation, etc.) ultimately depends on the data stored here.

Think of it as:

“The brain + database of your language system.”

Overall Purpose

This asset stores:

- Which languages exist
- Which language is currently active
- Where language files are located
- Which fonts should be used
- All translation data (texts + metadata)
- What to do when something goes wrong

It doesn't perform logic itself — it **provides structured data for other systems to use**.

Section 1: Folder and Language Settings

◆ folderName

What it is:

A string representing the folder where language files are stored.

Purpose:

Used to build file paths when loading language data.

Example:

If set to "Languages", your system may look for:

GameData/Languages/en-US/...

◆ defaultLanguage

What it is:

The fallback language.

Purpose:

If:

- No saved preference exists, or
- The system language is unsupported

Then this language is used.

Example:

"defaultLanguage = en"

Means English is the fallback.

Section 2: Font Configuration

Localization is not just text — fonts matter a lot.

◆ fontListData

What it is:

A reference to a font list for **standard UI elements**.

Purpose:

Stores fonts used by non-TextMeshPro systems.

◆ fontListDataTMP

What it is:

A reference to a font list specifically for **TextMeshPro**.

Purpose:

Used by scripts like your TMP localization component to:

- Swap fonts per language
- Support different alphabets

🖥️ Section 3: Canvas Configuration

◆ errorLanguageTool

What it is:

A reference to a UI object (a prefab).

Purpose:

Displayed when something goes wrong, such as:

- Language files missing
- Access denied
- Corrupted data

Example usage:

Your earlier permission checker uses this to show warnings.

📊 Section 4: Extracted Data (Core Runtime Data)

This is the **most important part** — the actual localization content.

◆ selectedCulture

What it is:

The currently active language.

Format:

Usually something like:

- "en-US"
- "pt-BR"
- "ja-JP"

Purpose:

All systems use this to decide:

- Which text to load
- Which audio to play
- Which assets to use

◆ availableLanguages

What it is:

A list of all languages defined in your system.

Each entry usually contains:

- Culture code
- Availability flag

Purpose:

Used to:

- Populate language selection menus
- Validate system language
- Filter supported languages

◆ idData

What it is:

A list of text entries mapped by ID.

Concept:

Each entry represents:

ID → Translated Text

Example:

ID: 10 → "Play"

ID: 10 → "Jogar"

Purpose:

This is how your system retrieves translated text:

- UI
- 3D text
- files
- audio references

◆ **idMetaData**

What it is:

Additional data linked to each ID.

What it can include:

- Font size
- Font index
- Special formatting info

Purpose:

Allows localization to affect **presentation**, not just content.

Example:

- Japanese text might need smaller font size
- Arabic text might need a different font

◆ **idCanvasData**

What it is:

A separate list of text IDs specifically for **Canvas UI elements**.

Why separate?

Canvas UI may:

- Use different systems
- Require different handling than 3D/world text

How the System Uses This Asset

Here's how everything connects:

Step 1 — Load Settings

Some system (like your initialization script) loads this asset.

Step 2 — Determine Language

- Check saved preference
- Otherwise use system language
- Otherwise fallback to default

Result → selectedCulture

Step 3 — Provide Data to Other Systems

Other scripts query this asset:

Text systems:

- Ask for text using ID → get from idData

TMP systems:

- Ask for font → get from fontListDataTMP
- Ask for metadata → get from idMetaData

Audio systems:

- Build file paths using selectedCulture

File creation:

- Use translated text from idData

Step 4 — Apply Changes

Scripts update:

- UI text
- Fonts
- Audio
- Files

All based on this asset.

Real Example Flow

Let's say the user selects Portuguese:

1. selectedCulture = "pt-BR"
2. A UI element asks:
 - "What is ID 5?"
3. System checks:
 - idData → finds "Jogar"
4. TMP system asks:
 - "What font should I use?"
5. System checks:
 - idMetaData → maybe returns font index 2
6. Font is retrieved from:
 - fontListDataTMP
7. UI updates:
 - Text = "Jogar"
 - Font = Portuguese-compatible font

Mental Model

Think of this script as:

A structured localization database

It contains:

- Configuration (where things are, defaults)
- Content (texts, metadata)
- References (fonts, UI prefabs)

Everything else in your system is just:

👉 “reading and applying” this data

⚠ Important Notes

- This script does **not execute behavior**
- It only stores data
- If something is wrong here, the entire localization system breaks

✅ When You Use This

You use this asset when:

- Setting up localization
- Adding new languages
- Adding new text IDs
- Assigning fonts
- Configuring fallback behavior

Adjust Size To Dropdown / Adjust Size To Dropdown TMP

This script is a **dynamic UI layout controller** designed specifically for dropdown-like interfaces that use TextMeshPro.

Its job is to:

- Detect when text becomes too long (wraps into multiple lines)
- Expand the UI element to fit that content
- Prevent the UI from going خارج the screen (canvas)
- Improve scrolling behavior automatically

🧠 Overall Concept

Think of this component as:

👉 **“A smart container that resizes and repositions itself based on its text content.”**

It constantly watches the text inside it and reacts in real time.

📦 Nested Structure

🔹 BrokenTextsTMP

What it represents:

A small data structure used to track each text element.

Contains:

- **itemLabel** → the text element being observed
- **brokenText** → whether the text has wrapped into multiple lines

Purpose:

Instead of checking text blindly, the script tracks:

- Which texts are “normal”
- Which ones are “overflowing”

Fit Direction System

◆ FitDirection (Left / Right)

What it controls:

The direction in which the UI expands.

Behavior:

- **Right** → expands to the right
- **Left** → expands to the left

Why it matters:

If the UI hits the edge of the screen:

- It flips direction automatically to stay visible

Settings (Manual Control)

◆ manualSizeAdjustment

Extra width added manually.

👉 Use this when automatic sizing isn't enough.

◆ sizeMultiplier

Extra width added **only when text breaks into multiple lines**.

👉 This is the “penalty” for long text.

◆ **margin**

Minimum distance from the edge of the canvas.

👉 Prevents UI from touching screen borders.

◆ **fitDirection**

Initial expansion direction.

👉 The script may override this dynamically later.

🤖 **Automatic Variables (Runtime Data)**

These are updated automatically while the game runs.

◆ **parentRect**

The UI element being resized.

◆ **canvasRectTransform**

The root canvas used to:

- Detect boundaries
- Prevent overflow

◆ **scrollRect**

Controls scrolling behavior.

◆ **canvasWidth**

Current width of the canvas.

◆ **objectWidth**

Current width of the UI element.

◆ **textList**

List of all text elements inside the object.

Each entry tracks:

- The text component
- Whether it wrapped

◆ **textIsBroken**

True if **any** text element wrapped.

◆ **sizeAdjustment**

Final calculated size increase.

Initialization Phase

◆ **Start**

This runs once when the object appears.

What it does:

1. Gets required components

- Finds its own UI container
- Finds scrolling system

2. Finds the Canvas

It walks up the hierarchy until it finds the canvas.

👉 This is critical for boundary detection.

3. Collects all text elements

- Finds every text component inside
- Stores them in the list
- Forces them to **overflow instead of clipping**

4. Adjusts scroll sensitivity

The more items you have:

- The faster scrolling becomes

But it is clamped to stay usable.

5. Scrolls to selected item

Automatically positions the list so:

👉 The selected option is visible

🎯 Scroll Logic

◆ ScrollToSelected

What it does:

- Finds which item is currently selected
- Moves the scroll position to center it

How:

- Calculates normalized position ($0 \rightarrow 1$)
- Applies it to the scroll system

🔄 Continuous Update

◆ Update

This runs every frame and is the **core of the system**.

Step 1: Safety Check

If no text exists → stop and warn

Step 2: Cache sizes

- Gets canvas width
- Gets object width

Step 3: Detect line breaks

Calls another method to check:

👉 Did any text wrap?

Step 4: Adjust size

If any text wrapped:

- Increase size using multiplier

Then add:

- manual adjustment

Step 5: Apply resizing

Depending on direction:

- Expand left OR right

Step 6: Convert position

Transforms object position into canvas space

👉 Needed to detect overflow correctly

Step 7: Boundary detection

If going خارج the right side:

- Switch direction → expand left

If going خارج the left side:

- Switch direction → expand right

Line Break Detection

◆ CheckLineBreaks

What it does:

Checks each text element:

- If it has more than 1 line → mark as broken

Why:

Line breaks mean:

👉 The container is too small

Extra:

Logs a warning when text wraps

Full Behavior Summary

1. Collect all text elements
2. Monitor them every frame
3. If any text wraps:
 - Increase width
4. Apply expansion:
 - Left or right
5. If UI hits screen edge:
 - Flip direction
6. Adjust scrolling:
 - Based on number of items
7. Auto-focus selected item

Practical Usage

Use this when you have:

Dropdowns with dynamic text

- Different languages
- Variable text length

Localization-heavy UI

Some languages are longer:

- German → longer words
- Japanese → compact but complex
- Russian → wider characters

Scrollable lists

Ensures:

- Selected item is always visible
- Scrolling feels natural

Important Observations

Continuous resizing

Runs every frame → can be slightly expensive if overused

Depends on proper hierarchy

Needs:

- Canvas above it
- Toggle system for selection

Assumes TMP usage

Won't work correctly with standard text components

Mental Model

Imagine this component as:

A flexible container that “breathes” based on its content

- Text grows → container grows
- Screen edge reached → container flips
- List grows → scrolling adapts

Language Canvas

This script is the **bridge between your language system and the visual layout of a Canvas**. While most localization systems only change text, this one also changes **how the UI is positioned, scaled, and structured depending on the language**.

Below is a clear breakdown of how everything works.

General Idea

Different languages often need different layouts.
For example:

- German text is longer → UI might need to stretch
- Japanese text is compact → UI might shrink
- Arabic uses right-to-left → layout may flip

This script solves that by:

1. Loading layout data tied to a language
2. Applying that layout dynamically to the Canvas
3. Allowing you to **save layouts in the editor**

Variables (Fields)

canvasID

- A unique identifier for this Canvas.
- Works like a **key** to find the correct layout data.
- Each Canvas in your game should have a different ID.

 Think:

“Which layout belongs to this Canvas?”

◆ canvasStructure

- Stores the full layout data of the Canvas.
- Includes things like:
 - Position of elements
 - Scale
 - Hierarchy (children structure)
 - UI configuration

👉 This is basically a **snapshot of the Canvas layout**.

🔒 Properties (Accessors)

These just allow other systems to safely read or modify:

- CanvasID → the identifier
- CanvasStructure → the stored layout data

No special logic here—just controlled access.

🔄 Unity Lifecycle Methods

◆ OnEnable

- Subscribes this component to a **global language change event**
- Immediately updates the Canvas to match the current language

👉 Result:

- When the object appears → it already uses the correct layout

◆ OnDisable

- Removes the subscription to avoid errors or memory leaks

👉 Important for stability when objects are destroyed or disabled

🌐 Core Logic

◆ LanguageUpdate

This is the **main entry point** when language changes.

What it does:

1. Loads the current language settings
2. Finds the layout data using the Canvas ID
3. Sends that data to be applied

👉 Flow:

Get current language

→ Find matching canvas layout (by ID)

→ Apply layout

◆ LoadCanvasData

This method receives layout data in **JSON format** and applies it.

Step-by-step:

1. Checks if the data exists
 - If empty → error
2. Converts JSON into a structured object
 - Turns raw text into usable layout data
3. Stores it in canvasStructure
4. Applies the layout to the actual Canvas

👉 This is where:

- Data becomes visual changes

✖ External Systems Used

This script relies on other systems:

◆ Language system

- Provides:
 - Current language
 - Canvas layout data linked by ID

◆ Canvas manager

- Responsible for:
 - Applying layout to GameObjects

- Rebuilding UI structure

👉 This script does NOT apply layout directly—it delegates that job.

🔧 Editor-Only Functionality

◆ SaveCanvasData

Used only during development.

What it does:

1. Reads the current Canvas layout
2. Converts it into structured data
3. Serializes it into JSON

👉 This allows you to:

- Design UI visually
- Save it as language-specific layout data

🔧 Custom Inspector (Editor Tool)

Adds a button:

◆ “Import Settings”

When clicked:

1. Checks if the ID already exists
2. Asks for confirmation (to avoid overwriting)
3. Saves the current layout
4. Opens an editor window for further editing

👉 This is your workflow:

Design UI → Click Import → Save as language layout

📊 Full Runtime Flow

Here's what happens in a real scenario:

● Game starts or language changes:

1. Event triggers LanguageUpdate

2. Script loads language settings
3. Finds layout using canvasID
4. Converts JSON → layout structure
5. Applies layout to Canvas

In the Editor:

1. You design your Canvas visually
2. Click "Import Settings"
3. Layout gets saved as JSON
4. Linked to a language via ID

Usage

How to use this script:

1. Attach it to a Canvas object
2. Assign a unique canvasID
3. Design your layout
4. Save it using the editor tool
5. Repeat for different languages

Example Use Cases

- UI repositioning for different languages
- Right-to-left layout switching
- Different UI scaling per region
- Platform-specific UI adjustments

Important Notes

- If JSON data is missing → layout will not update
- Canvas must exist and be active
- IDs must match language data

Key Insight

This system goes beyond simple translation:

 It localizes **structure**, not just text.

That's a big deal because:

- Many localization systems break UI
- This one adapts the UI itself

Language Dropdown / Language Dropdown TMP

This script is responsible for **localizing a dropdown UI element**—not just the text, but also its **visual style (font, size, alignment)** based on the selected language.

Think of it as a smart adapter between:

- your **language system**
- and a **dropdown menu**

General Idea

A dropdown usually contains:

- A list of options (Option A, B, C...)
- A selected value (caption)
- A visual style (font, alignment, size)

This script ensures that **all of that adapts dynamically to the current language**.

Variables (Fields)

◆ dropdown

- Reference to the dropdown UI element.
- This is the main component being controlled.

👉 If this is missing → nothing works.

◆ translateText

- A toggle that decides:
 -  Translate option texts
 -  Keep original texts

👉 Useful when:

- You want icons only
- Or already-localized content

◆ options

- A list of dropdown entries.
- Each entry contains:
 - Text (default/fallback)
 - Image (optional icon)
 - ID (used for translation)

👉 This is your **data source** for the dropdown.

🔒 Private Variables

◆ languageData

- Stores all loaded language information:
 - Translations
 - Metadata (font, size, alignment)

◆ captionText

- The visible text showing the selected option.

◆ itemText

- The template text used for each dropdown item.

👉 These are extracted from the dropdown automatically.

🔄 Unity Lifecycle

◆ OnEnable

- Subscribes to the global language change event
- Immediately updates the dropdown

👉 Ensures correct language from the start

◆ OnDisable

- Unsubscribes from the event

👉 Prevents errors and memory issues

🌐 Core Logic

💠 LanguageUpdate

This is the **main function** that updates everything.

Step-by-step:

1. Validation

- Checks if:
 - Dropdown exists
 - Caption text exists
 - Item text exists

👉 If something is missing → stops with error

2. Load language data

- Gets:
 - Translations
 - Style metadata

👉 Without this → nothing can be localized

3. Validate options

- Ensures there are items to display

4. Translate options (if enabled)

- Calls UpdateLocalizedOptions

5. Apply style (from metadata)

It uses the **first option's ID** to define style for everything.

Why?

👉 Because dropdown style is shared across all items.

Applies:

► **Alignment**

- Left / Center / Right

► **Font size**

- Makes text bigger or smaller depending on language

► **Font**

- Switches fonts (important for languages like Japanese, Arabic, etc.)

 Option Translation

◆ **UpdateLocalizedOptions**

This method handles the **text replacement process**.

Step-by-step:

1. Translate each option

- Uses the ID to fetch translated text
- Replaces the original text if found

2. Save current selection

- Stores which option is selected

👉 Important to avoid resetting user choice

3. Clear dropdown

- Removes all current options

4. Rebuild dropdown

- Adds updated (translated) options back

5. Restore selection

- Reapplies the previously selected index
- Updates visible caption

👉 Result: seamless update without user noticing disruption

🌿 Metadata System

The script uses **metadata tied to IDs** to control visuals.

From metadata it gets:

- Alignment
- Font size
- Font index

👉 This allows:

- Per-language styling
- Per-text customization

🔧 Editor Tools

◆ Custom Inspector

Adds an “**Import Settings**” button.

What happens when clicked:

1. Checks if IDs already exist
2. Asks for confirmation if needed
3. Extracts current style from dropdown:
 - Alignment
 - Font size
 - Font

4. Opens an editor window

Behavior:

- First option → includes style data
- Other options → only text

👉 This keeps styling consistent across the dropdown

Runtime Flow

● When game starts or language changes:

Event triggered

→ LanguageUpdate()

→ Load language data

→ Translate options

→ Apply font + size + alignment

→ Refresh dropdown UI

Usage

How to use this script:

1. Attach it to a GameObject
2. Assign a dropdown component
3. Fill the options list with:
 - Default text
 - IDs for translation
4. Configure language data system
5. Done — it updates automatically

Example Use Case

Imagine this dropdown:

Option English Portuguese

A Play Jogar

B Settings Configurações

C Exit Sair

This script:

- Replaces text automatically
- Adjusts font if needed
- Keeps selection intact

Important Notes

- The **first option controls style**
- IDs must exist in language data
- Missing TMP components = errors
- Rebuild happens every language update

Key Insight

This script doesn't just translate text.

 It ensures the dropdown:

- Looks correct
- Feels consistent
- Adapts to different languages visually

Language Image

This script is responsible for **changing UI images based on the selected language**. Instead of only translating text, it allows you to **swap entire images (sprites)** depending on the player's language.

General Idea

Different languages may require different images. For example:

- Text inside an image (like a button label)
- Cultural differences (icons, symbols)
- Region-specific assets

This script:

1. Detects the current language
2. Builds the correct file path
3. Loads the image from disk

4. Applies it to a UI Image component

Variables (Fields)

◆ **fileName**

- The name of the image file to load
- Must include extension (like .png, .jpg)

👉 Example:

PlayButton.png

◆ **useImage**

- Enables or disables applying the image to the UI component

👉 Useful if:

- You want to load but not display
- Or temporarily disable updates

◆ **image**

- Reference to the UI Image element on screen

👉 This is where the final sprite will be shown

◆ **imageTexture**

- Stores the raw image data loaded from disk

👉 Think of it as:

- The “pixel data” before becoming a sprite

◆ **spriteTexture**

- The final usable image for UI

👉 Unity UI uses sprites, not raw textures

🔒 Private Variables

◆ languageData

- Contains current language info:
 - Selected culture (like en, pt-BR, etc.)

◆ previousFilePath

- Stores the last loaded file path

👉 Used to avoid reloading the same image repeatedly

◆ filePath

- The full path to the image file

👉 Built dynamically based on language

🔄 Unity Lifecycle

◆ OnEnable

- Subscribes to the global language change event
- Immediately updates the image

👉 Ensures correct image appears right away

◆ OnDisable

- Unsubscribes from the event

👉 Prevents unwanted calls or memory issues

🌐 Core Logic

◆ LanguageUpdate

This is the **main function** that updates the image.

Step-by-step:

1. Validate Image component

- If useImage is enabled but no Image is assigned → error

2. Load language settings

- Gets the current language (culture)

3. Build file path

The path follows a structure like:

[LanguageAssetsPath]/Assets/[Culture]/Image/[FileName]

👉 Example:

.../Assets/en/Image/PlayButton.png

.../Assets/pt-BR/Image/PlayButton.png

4. Avoid unnecessary reload

- If the path is the same as last time → do nothing

👉 Optimization to save performance

5. Check if file exists

- If missing → error

6. Clean previous data

- Removes old texture and sprite from memory

👉 Prevents memory leaks

7. Start asynchronous loading

- Calls the loading process (non-blocking)

⚙️ Asynchronous Loading

◆ LoadFileCoroutine

This is responsible for **actually loading the image**.

Step-by-step:

1. Request image from disk

- Uses a file-based request system

👉 Works even outside the editor (runtime)

2. Wait for loading

- Doesn't freeze the game while loading

3. Handle errors

- If loading fails → logs error

4. Extract texture

- Converts downloaded data into a texture

5. Create sprite

- Converts texture into a UI-compatible sprite

6. Apply to UI

- Assigns sprite to the Image component

Custom Inspector (Editor)

Adds a simple warning:

File name restriction

- Advises using only basic ASCII characters

 Avoids problems with:

- File paths
- Encoding issues
- Cross-platform compatibility

Runtime Flow

When game starts or language changes:

LanguageUpdate()

→ Get current language

→ Build file path

→ Check if changed

→ Load image asynchronously

→ Convert to sprite

→ Apply to UI

Usage

How to use:

1. Attach this script to a GameObject
2. Assign a UI Image component
3. Set the file name (e.g., Logo.png)
4. Organize your files like:

Assets/

en/

Image/

Logo.png

pt-BR/

Image/

Logo.png

5. Done — it updates automatically when language changes

💡 Example Use Cases

- Localized logos
- Buttons with embedded text
- Region-specific icons
- Tutorial images per language

⚠️ Important Notes

- File must exist in the correct language folder
- File name must match exactly
- Reload only happens when path changes
- Old textures are destroyed to save memory

🧠 Key Insight

This script treats images like text:

👉 Instead of translating strings, it **swaps assets dynamically**.

Language Raw Image

This component is very similar in purpose to the previous one, but instead of working with a standard UI Image (which uses Sprites), it works with a **RawImage**, meaning it displays a **texture directly**. That makes it useful for cases like dynamic images, videos, or non-sliced visuals.

Let's break it down clearly.

Overall Purpose

This script:

- Loads an image file from disk based on the **current language**
- Converts it into a **texture**
- Assigns that texture to a **RawImage UI element**
- Updates automatically whenever the language changes

Variables (What each one does)

Settings

fileName

- The name of the image file to load (example: "flag.png")
- Must include the extension
- This is the only thing you change to point to a different image

useRawImage

- A simple ON/OFF switch
- If enabled → the RawImage component will be updated
- If disabled → the script loads nothing into the UI

rawImage

- Reference to the UI element that will display the image
- This is where the loaded texture will appear

imageTexture

- Stores the loaded texture from disk
- This is the raw image data

Internal (Private) Variables

languageData

- Holds the current language configuration

- Includes things like selected language (e.g. "en-US", "pt-BR")

previousFilePath

- Stores the last loaded path
- Used to avoid reloading the same image unnecessarily

filePath

- The full path to the image file
- Built dynamically based on:
 - base folder
 - selected language
 - image name

Example result:

.../Languages/Assets/en-US/Image/flag.png

Lifecycle (Automatic Behavior)

OnEnable

- Subscribes to a global "language changed" event
- Immediately updates the image

👉 This ensures the UI is correct as soon as it appears

OnDisable

- Stops listening to language changes
- Prevents unnecessary processing or memory leaks

Main Logic

LanguageUpdate

This is the heart of the system.

Step-by-step:

1. **Check RawImage**

- If it's required but missing → error

2. Load language settings

- Retrieves current selected language

3. Build file path

- Combines:
 - base language folder
 - "Assets"
 - selected language
 - "Image"
 - file name

4. Avoid redundant reload

- If path hasn't changed → do nothing

5. Check if file exists

- If not → error

6. Clean previous texture

- Prevents memory leaks

7. Start asynchronous loading

- Calls the coroutine

LoadFileCoroutine (Async Loader)

This runs in the background.

Steps:

1. Request the image file

- Loads from local disk using a web-style request

2. Wait until loading finishes

3. Handle errors

- If something fails → log error

4. Extract texture

- Converts file into usable texture data

5. Apply to RawImage

- If enabled → assign texture to UI

Important Behavior

No Sprite Creation

Unlike the Image version:

- This script does **NOT create a sprite**
- It uses the texture directly

 That's why it works with RawImage

Performance Optimization

- Avoids reloading the same file
- Destroys old textures before loading new ones
- Uses async loading (non-blocking)

Usage (How to use it)

1. Add to a GameObject

Attach this component to any UI object

2. Assign RawImage

- Drag your RawImage component into the field

3. Set file name

Example:

logo.png

background.jpg

flag_en.png

4. Organize your files like this:

Languages/

└─ Assets/

 └─ en-US/

 └─ Image/

 └─ logo.png

 |

 └─ pt-BR/

 └─ Image/

 └─ logo.png

5. Change language at runtime

When the language changes:

- The script automatically loads the correct image
- UI updates instantly

When to Use This vs Image Version

Use this (RawImage) when:

- You want **full texture control**
- You don't need sprite slicing
- You're displaying:
 - backgrounds
 - dynamic images

- video frames
- large textures

Use Image version when:

- You need **UI sprites**
- You rely on:
 - slicing
 - borders
 - UI layout features

⚠ Common Pitfalls

- ❌ Forgetting to assign RawImage
- ❌ Wrong file name or extension
- ❌ Missing language folder structure
- ❌ Using special characters in file names
- ❌ Large textures causing memory spikes

✚ Summary

This script acts like a **language-aware image loader for RawImage**:

- Builds a path based on current language
- Loads texture asynchronously
- Updates UI automatically
- Prevents redundant work
- Cleans memory properly

Language Manager / Language Manager TMP

This script is essentially the **control center for language selection using a dropdown UI**. It connects your localization system with the user interface, allowing players to switch languages at runtime—and making sure everything else updates accordingly.

Let's break it down clearly.

🧠 Overall Purpose

This component:

- Loads all available languages from your localization data
- Displays them in a dropdown list

- Restores the previously selected language
- Lets the user change the language
- Saves the selection
- Notifies the entire system to update texts, images, audio, etc.

👉 In short: **this is the user-facing language selector**

⚙ Variables (What each one does)

🔑 UI Component

languageDropdown

- The dropdown UI element the user interacts with
- Displays all available languages
- Handles user selection

🔒 Internal Variables

availableLanguages

- A list of all supported languages
- Comes from the localization settings
- Each entry typically contains:
 - display name
 - native name (e.g., "Português", "English")
 - culture code (e.g., "pt-BR", "en-US")

languageData

- The main configuration object
- Contains:
 - available languages
 - current language
 - all localization data

🔄 Lifecycle

Start

This is where everything begins.

Step-by-step:

1. Check if dropdown exists

- If not → error

2. Load language settings

- Retrieves all localization data

3. Get available languages

- Fills the internal list

4. Populate dropdown

- Adds language names to the UI

5. Setup event listener

- Prepares the dropdown to react to user changes



Dropdown Population

PopulateDropdown

This method prepares what the user sees.

What it does:

1. Clear existing options

- Ensures no duplicates

2. Get saved language

- Retrieves previously selected language from storage

3. Sort languages

- Alphabetically (by name)

4. Fill dropdown list

- Uses the *native name* (important for UX)

Example:

- English
- Português
- Español

5. Find selected index

- Matches saved language with list

6. Apply selection

- Sets dropdown without triggering events

👉 This ensures:

- The UI reflects the user's last choice
- No unnecessary updates are triggered

Selection Handling

SetupDropdownSelection

- Removes old listeners (prevents duplicates)
- Adds a new listener for when the user changes selection

OnLanguageChanged

This is the most important interaction point.

Trigger:

When the user selects a different language

Step-by-step:

1. Validate index

- Prevents invalid selections

2. Get selected language

- Retrieves culture code (e.g., "en-US")

3. **Save selection**

- Stores it for future sessions

4. **Reload language data**

- Updates internal localization system

5. **Notify the system**

- Broadcasts a global "language changed" event

System Integration

This script works together with all your other components:

When **OnLanguageChanged** runs:

👉 It triggers a global update

Then all components like:

- text localizers
- image loaders
- audio players
- canvas layouts

...automatically refresh themselves

Flow Summary

When the game starts:

1. Load language data
2. Populate dropdown
3. Restore saved language

When user changes language:

1. Save new language
2. Reload localization data
3. Notify entire system

4. Everything updates instantly

Usage (How to use it)

1. Add to a GameObject

Attach this component anywhere in your UI (usually near menus)

2. Assign Dropdown

- Link your TMP dropdown component

3. Configure Language Data

Make sure your language settings include:

- available languages
- culture codes
- translations

4. Run the game

You'll get:

- A dropdown with languages
- Automatic selection restore
- Instant language switching

Design Strengths

User-friendly

- Uses native language names
- Automatically sorted

Persistent

- Remembers user choice

✅ Decoupled system

- Uses global event → no tight dependencies

✅ Scalable

- Just add new languages → dropdown updates automatically

⚠️ Common Pitfalls

- ❌ Dropdown not assigned
- ❌ No languages in settings
- ❌ Missing culture codes
- ❌ Not saving language properly
- ❌ Other components not listening to update event

🧩 Conceptual Summary

Think of this script as:

🕒 **A controller that connects the player's choice to the entire localization system**

It:

- Displays languages
- Handles user input
- Saves preferences
- Triggers global updates

Language Text / Language Text TMP

This component is one of the **core building blocks of your localization system**. It's responsible for taking a normal TextMeshPro text element and making it **fully language-aware**, not just changing the text, but also adapting how it looks.

🧠 Overall Purpose

This script:

- Connects a TMPro text element to your localization system

- Replaces its content based on the selected language
- Adjusts visual properties like:
 - alignment
 - font
 - font size
- Updates automatically whenever the language changes

👉 Think of it as a **smart text component that reacts to language changes**

⚙️ Variables (What each one does)

🔧 UI Components

textComponent

- The actual text element shown on screen
- This is what gets modified (text, font, alignment, size)

translateText

- A toggle (true/false)
- If enabled → text will be replaced with translated content
- If disabled → text stays as-is, but styling can still change

👉 Useful when:

- You want dynamic styling but fixed text

iD

- The most important value
- A unique identifier that links this text to the localization database

👉 This ID is used to:

- Find the translated text
- Find metadata (font, size, alignment)

🔒 Internal Variable

languageData

- Holds the currently loaded language configuration
- Contains:

- all translations
- all metadata (font, alignment, etc.)

Lifecycle

OnEnable

When the object becomes active:

1. Subscribes to the global “language changed” event
2. Immediately updates the text

👉 This guarantees correct language on startup

OnDisable

- Stops listening to language updates
- Prevents unnecessary processing

Main Logic

LanguageUpdate

This is the core method where everything happens.

Step-by-step:

1. Validate text component

- If missing → error
- Prevents crashes

2. Load language settings

- Retrieves current language data
- Includes translations + metadata

3. Apply translated text (optional)

If **translateText** is enabled:

- Look up text using the ID
- If found → replace current text

👉 Example:

- ID = 10
- English → "Start Game"
- Portuguese → "Iniciar Jogo"

4. Apply metadata (visual adjustments)

The script doesn't stop at text—it also adjusts how it looks.

◆ Alignment

- Changes how text is positioned (left, center, right, etc.)
- Based on stored metadata

◆ Font Size

- Adjusts size per language
- Important because:
 - some languages are longer
 - some need bigger/smaller text

◆ Font

- Switches font dynamically
- Critical for:
 - different alphabets (Latin, Cyrillic, Asian, etc.)
 - readability

🧠 Why Metadata Matters

This is where your system becomes powerful.

Instead of just translating text, you also control:

- layout behavior
- visual consistency

- language-specific readability

👉 Example:

English:

- Short words → smaller font

German:

- Long words → smaller font or different alignment

Japanese:

- Different font entirely

🔄 Full System Flow

When the game starts:

1. Component activates
2. Loads language data
3. Applies text + style

When language changes:

1. Global event is triggered
2. This script receives it
3. Runs LanguageUpdate again
4. UI updates instantly

🔧 Usage (How to use it)

1. Add to a GameObject

Attach this component to any object with a TextMeshPro text

2. Assign Text Component

- Drag your TMP text into the field

3. Set ID

- Must match an entry in your language system

4. Configure Translate Toggle

- ON → full localization
- OFF → styling only

5. Run the game

Result:

- Text updates automatically
- Style adapts per language

Practical Examples

UI Button

- “Play”, “Options”, “Exit”

Dialogue System

- NPC conversations

HUD Elements

- Score, health, messages

Dynamic Labels

- Tooltips
- Menus

Common Pitfalls

-  Text component not assigned

- ❌ Invalid or missing ID
- ❌ No translation for that ID
- ❌ Metadata not configured
- ❌ Fonts missing for certain languages

✖ Design Strengths

✅ Fully automatic

- No manual updates needed after setup

✅ Centralized control

- Everything driven by IDs

✅ Scalable

- Add new languages without changing code

✅ Flexible

- Can separate translation and styling

🧠 Conceptual Summary

This script is:

✖ A bridge between your UI text and your localization database

It:

- Finds the correct translation
- Applies it to the UI
- Adjusts visual presentation
- Reacts to language changes instantly

Language Text Input Field / Language Text Input Field TMP

This component is a **specialized version of your text localization system**, designed specifically for **input fields**. Instead of just handling a single text element, it manages **two parts at once**:

- The **placeholder text** (the hint shown before typing)
- The **user input text style** (font, size, alignment)

Overall Purpose

This script:

- Localizes the **placeholder text** of an input field
- Applies visual settings (font, size, alignment) to both:
 - placeholder text
 - user-typed text
- Updates automatically when the language changes

👉 Think of it as:

✳️ “LanguageTextTMP, but adapted for input fields with extra care for UX.”

Variables (What each one does)

UI Components

inputField

- The main input field element
- Contains:
 - placeholder text
 - actual typed text

translateText

- Enables/disables automatic translation
- If enabled → placeholder gets translated
- If disabled → placeholder stays unchanged

iD

- Unique identifier used to fetch:
 - translated placeholder text
 - metadata (font, alignment, size)

Internal Variables

languageData

- Stores the current language configuration
- Provides access to:
 - translations
 - styling metadata

text

- The text component where the user types
- This is NOT the placeholder

placeholder

- The hint text shown when input is empty
- This is what gets translated

Lifecycle

OnEnable

When the object becomes active:

1. Subscribes to global language update event
2. Immediately applies localization

👉 Ensures correct language from the start

OnDisable

- Stops listening to language updates
- Prevents unnecessary processing

Main Logic

LanguageUpdate

This is the core method that updates everything.

Step-by-step breakdown:

1. Validate Input Field

- If missing → error
- Prevents crashes

2. Get Placeholder Component

- Extracts the text element used as placeholder
- Must be a compatible text type

3. Get Input Text Component

- Extracts the actual typing text element
- This is what the user sees while typing

4. Load Language Data

- Retrieves current language configuration

5. Translate Placeholder (optional)

If translation is enabled:

- Look up text using the ID
- Replace placeholder text if found

👉 Example:

- English → “Enter your name”
- Portuguese → “Digite seu nome”

⚠ Important:

- Only the **placeholder is translated**
- The user’s typed text is NEVER replaced

6. Apply Metadata (visual styling)

This affects BOTH:

- placeholder text
- input text

Styling Adjustments

◆ Alignment

- Changes how text is positioned
- Applied to both placeholder and input text

◆ Font Size

- Ensures readability across languages
- Some languages require more space

◆ Font

- Allows switching fonts per language
- Critical for:
 - special characters
 - different alphabets

Why This Matters

Input fields are tricky because:

- Placeholder must be localized
- User input must NOT be overwritten
- Style must remain consistent

👉 This script handles all three correctly.

Full System Flow

On start:

1. Component activates
2. Loads language
3. Updates placeholder + styling

On language change:

1. Event is triggered
2. This script updates again
3. Placeholder + style change instantly



Usage (How to use it)

1. Add to a GameObject

Attach it to the same object as your input field

2. Assign Input Field

- Drag your TMP_InputField into the field

3. Set ID

- Must match a translation entry

4. Enable/Disable Translation

- ON → localized placeholder
- OFF → keep original placeholder

5. Run the game

Result:

- Placeholder changes with language
- Text styling adapts automatically
- User input remains untouched

Practical Examples

Login Screen

- “Username”
- “Password”

Search Bars

- “Search...”
- “Type here...”

Forms

- “Enter email”
- “Phone number”

Common Pitfalls

-  InputField not assigned
-  Placeholder is not a compatible text type
-  Missing ID in language data
-  No metadata configured
-  Font missing for certain languages

Design Strengths

Safe for user input

- Never overwrites typed content

Consistent styling

- Placeholder and input match visually

Fully automatic

- Updates on language change

✅ Scalable

- Works with any number of languages

🧠 Conceptual Summary

This script is:

✂️ **A localization layer for input fields that respects both UX and language rules**

It:

- Translates placeholder text
- Keeps user input intact
- Applies consistent styling
- Reacts to language changes globally

Automatic Language Font Validator / Automatic Language Font Validator TMP

This component is a **safety system for fonts**. Its job is to make sure that whatever text is being shown on screen can actually be rendered by the current font. If not, it tries to fix the problem automatically.

🧠 Overall Idea

Different languages use different characters. For example:

- English → simple Latin letters
- Japanese → kanji + kana
- Arabic → completely different script

Not every font supports all of these.

So this script:

1. Watches the text constantly
2. Checks if the current font supports every character
3. If not → tries other fonts
4. If still not → falls back to a safer text (optional behavior)

Variables Explained

UI Reference

textComponent

- The text element being monitored.
- This is the **actual UI text** shown to the player.
- Everything revolves around this.

Behavior Control

isLanguageManager

- Enables a **special fallback behavior**.
- If turned ON:
 - When a language name (like “日本語”) cannot be displayed,
 - It replaces it with a simpler version (like “Japanese”).

This is especially useful for **language selection menus**.

Internal Data

supportedLanguages

- A list of all available languages.
- Used only when fallback behavior is enabled.
- Helps map:
 - Native name → fallback name

localizationSettings

- The global language configuration.
- Contains:
 - Available languages
 - Font lists
 - Other localization data

lastValidatedText

- Stores the last checked text.
- Prevents unnecessary re-checking every frame.

👉 Important for performance.

⚙️ How It Works (Step-by-Step)

1. Initialization (Start)

When the object starts:

- Checks if the text component exists
- Loads language settings
- If fallback mode is enabled:
 - Loads available languages

Then it immediately validates the current text.

2. Continuous Monitoring (Update)

Every frame:

- Gets the current text
- Compares it with the previous one

If the text changed:

- Runs validation again

👉 This ensures updates happen **only when needed**, not constantly.

3. Core Logic (Validation)

This is the heart of the system.

Step A — Check Current Font

- Goes through each character in the text
- Asks the font:

“Can you display this character?”

If **all characters are supported**:

- Nothing happens ✓

Step B — Try Fallback Fonts

If the current font fails:

- It loops through a list of alternative fonts
- For each font:
 - Checks if it supports all characters

If one works:

- It replaces the font
- Stops the process

Step C — Last Resort (Optional)

If no font works:

- Only happens if `isLanguageManager` is ON

Then:

- It checks if the text matches a language name
- If yes:
 - Replaces it with a fallback name (usually simpler characters)

Example:

- "日本語" → "Japanese"

Why This Is Important

Without this system:

- Missing characters show as:
 - □ (squares)
 - ? (unknown symbols)
- UI becomes unreadable for some languages

This script prevents that automatically.

Practical Usage

Where to Use

1. Language Selection Menus

- Ensures all language names are readable
- Especially important for multilingual games

2. Dynamic Text Systems

- Chat systems
- User-generated content
- External/localized text files

3. Games Supporting Multiple Alphabets

- Japanese, Chinese, Korean
- Arabic
- Cyrillic languages

Typical Setup

1. Attach this component to a text element
2. Assign the text reference
3. Provide a list of fallback fonts in your settings
4. Enable fallback mode if needed

Important Notes

Performance

- Efficient because it only runs when text changes
- Still, avoid using it on hundreds of texts unnecessarily

Font Setup Matters

- The fallback list must be well configured
- Otherwise, it won't find a valid font

Not a Translator

- This script does **not translate text**
- It only ensures text can be displayed correctly

In Simple Terms

Think of it like this:

“If the font can’t draw the letters... find one that can.”

And if none can:

“At least show something readable.”

Force Import Of Fonts TMP

This script solves a very specific (and annoying) problem in Unity:

Sometimes fonts exist in your project... but **don’t get included in the final build**.

So even though everything works in the editor, in the final game:

- characters may disappear
- fallback fonts fail
- text becomes broken

This component forces Unity to **recognize and include every font** used in your localization system.

Core Idea

Unity only includes assets in the build if it detects they are being used.

This script “tricks” Unity by:

1. Creating temporary text objects
2. Assigning each font to one of them
3. Displaying sample text

👉 Result: Unity is forced to import and keep those fonts.

Variables Explained

Base Setup

textMesh

- A reference object used as a **template**.
- It must contain a text component.

- The script duplicates this object for each font.

👉 Think of it as a “prototype” text.

Custom Preview Texts

fontTexts

- Optional list that pairs:
 - a font
 - a custom sample text

Instead of using the font name as preview text, it can show something like:

- real characters from that language
- special symbols

👉 Useful to ensure glyphs are actually included.

Generated Objects

gameObjects

- Stores all the temporary objects created.
- Prevents duplication when re-running the script.
- Also allows cleanup before regenerating.

⚙️ How It Works (Step-by-Step)

1. Execution Mode

- Runs **only inside the editor**
- Never runs in the final game

If somehow it runs in runtime:

- It deletes itself immediately

2. Initialization

When triggered:

Step A — Validate Template

- Checks if the base text object exists
- Stops if missing

Step B — Clean Previous Objects

- Deletes all previously generated objects
- Avoids duplicates and clutter

Step C — Load Language Data

- Loads global localization settings
- Retrieves the list of fonts used

Step D — Validate Font List

- If no fonts exist → logs warning and stops

3. Font Processing Loop

For each font in the list:

Step A — Create a Clone

- Duplicates the template object
- Places it under this component

Step B — Get Text Component

- Ensures the clone actually has a text element
- If not → deletes it and skips

Step C — Decide Text Content

Default:

- Uses the font name

But if a custom entry exists:

- Uses something like:
"Text (FontName) - [Custom Sample]"

👉 This is important for testing real characters.

Step D — Apply Font

- Assigns the current font to the text

Step E — Apply Text

- Sets the preview text

Step F — Organize Object

- Renames it for clarity
- Activates it
- Stores reference in the list

4. Final Result

At the end:

- You have one text object per font
- Each one actively uses its font

👉 Unity now **guarantees those fonts are included in the build**

Why This Works

Unity's build system:

- Includes assets only if they are referenced
- Ignores unused assets

This script creates **real usage references**, so:

- Fonts are no longer “invisible” to the build system

Practical Usage

Where to Use

1. Initialization Scene

- Splash screen
- Main menu

This ensures fonts are loaded early.

2. Localization Systems

- Projects supporting multiple languages
- Especially those with:
 - Asian scripts
 - Special symbols

3. TMP Font Variants

- When using multiple font assets for fallback

Typical Setup

1. Create a simple text object (your template)
2. Assign it to this component
3. Configure optional sample texts (recommended)
4. Run the script (or press reload)

Important Details

Editor Only

- This is NOT a runtime system
- It exists purely to prepare the build

Scene Cleanup

- The generated objects stay in the scene
- Usually fine, but you may:

- hide them
- place them in a disabled container

Custom Text Is Powerful

If you use:

- Japanese fonts → include kanji samples
- Arabic fonts → include Arabic text

👉 This ensures glyphs are baked correctly.

🧠 In Simple Terms

This script says:

“Hey Unity, these fonts ARE being used. Don’t ignore them.”

And it proves it by:

- physically creating text objects using each font

💡 Extra Insight

Without this system, a common bug happens:

- Everything works in Editor ✓
- Build is missing characters ✗

This script eliminates that entire class of problems.